

داده کاوی

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم اول سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل دوازدهم: Data mining trends and research frontiers

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو و امیرحسین همتی

فهرست

- 12.1 Mining rich data types
 - 12.1.1 Mining text data
 - 12.1.2 Spatial-temporal data
 - 12.1.3 Graph and networks
- 12.2 Data mining applications
 - 12.2.1 Data mining for sentiment and opinion
 - 12.2.2 Truth discovery and misinformation identification
 - 12.2.3 Information and disease propagation
 - 12.2.4 Productivity and team science
- 12.3 Data mining methodologies and systems
 - 12.3.1 Structuring unstructured data for knowledge mining: a data-driven approach
 - 12.3.3 From correlation to causality
 - 12.3.4 Network as a context
 - 12.3.5 Auto-ML: methods and systems
- 12.4 Data mining, people, and society
 - 12.4.1 Privacy-preserving data mining
 - 12.4.2 Human-algorithm interaction
 - 12.4.3 Mining beyond maximizing accuracy: fairness, interpretability, and robustness
 - 12.4.4 Data mining for social good

Section 12.1

12.1.1. Mining text data

کاوش داده‌های متنی (Mining text data)

کاوش داده‌های متنی شامل استخراج الگوهای معنادار، بینش‌ها و دانش از اطلاعات متنی بدون ساختار است. این یک جنبه حیاتی از کاوش داده‌ها است، زیرا داده‌های متنی به اشکال مختلفی مانند مقالات، نقدها، پست‌های رسانه‌های اجتماعی و موارد دیگر فراوان هستند. تکنیک‌هایی مانند پردازش زبان طبیعی (Natural Language Processing - NLP) برای تحلیل و استخراج اطلاعات ارزشمند از محتوای متنی به کار گرفته می‌شوند.

نکات کلیدی درباره کاوش داده‌های متنی:

- قطعه‌سازی و پیش‌پردازش (Tokenization and Preprocessing): داده‌های متنی معمولاً از طریق قطعه‌سازی (tokenization) پردازش می‌شوند، که آن‌ها را به کلمات یا عبارات مجزا تجزیه می‌کند. مراحل پیش‌پردازش (preprocessing) ممکن است شامل حذف کلمات توقف (stop words)، ریشه‌یابی (stemming) و ریشه‌یابی صرفی (lemmatization) برای کاهش نویز و بهبود تحلیل باشد.

- **استخراج ویژگی (Feature Extraction):** تکنیک‌هایی مانند فراوانی واژه-معکوس فراوانی سند (Term Frequency-Inverse Document Frequency - TF-IDF) یا بردار تعبیه‌های واژه (word embeddings) مانند Word2Vec و GloVe برای تبدیل متن به ویژگی‌های عددی استفاده می‌شوند که می‌توانند در الگوریتم‌های کاوش داده‌ها به کار روند.
- **تحلیل احساسات (Sentiment Analysis):** تحلیل احساسات در متن یک کاربرد رایج است که لحن عاطفی یک نوشته را تعیین می‌کند. این برای درک نظرات مشتریان، نقدها و احساسات رسانه‌های اجتماعی ارزشمند است.
- **مدل‌سازی موضوعی (Topic Modeling):** شناسایی موضوعات در یک مجموعه بزرگ از اسناد متنی از طریق الگوریتم‌های مدل‌سازی موضوعی مانند Latent Dirichlet Allocation (LDA) به دست می‌آید. این امر به سازماندهی و دسته‌بندی محتوای متنی کمک می‌کند.
- **کاربردها (Applications):** کاوش متن کاربردهایی در زمینه‌های متنوعی مانند تحلیل رسانه‌های اجتماعی، بازخورد مشتریان، بازیابی اطلاعات و دسته‌بندی خودکار اسناد پیدا می‌کند.

In [3]: *NewsGroups روی مجموعه داده 20 LinearSVC آموزش: طبقه‌بندی متون با استفاده از #*

```
# 1 وارد کردن کتابخانه‌های مورد نیاز
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import LabelEncoder
from sklearn.datasets import fetch_20newsgroups

# 2 NewsGroups بارگذاری مجموعه داده متنی 20
newsgroups = fetch_20newsgroups(subset='all', remove=('headers', 'footers', 'quotes'))
df = pd.DataFrame({
    'text': newsgroups.data,
    'category': [newsgroups.target_names[i] for i in newsgroups.target]
})

# 3 تقسیم داده‌ها به آموزش و آزمون (محدود به ۵۰۰۰ نمونه برای سرعت بیشتر)
train_data, test_data, train_labels, test_labels = train_test_split(
    df['text'], df['category'], test_size=0.2, random_state=42
)

# 4 TF-IDF تبدیل متن به ویژگی‌های عددی با استفاده از
vectorizer = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1,2),
    stop_words='english',
    lowercase=True
)
X_train = vectorizer.fit_transform(train_data)
X_test = vectorizer.transform(test_data)

# 5 رمزگذاری برچسب‌ها به صورت عددی
label_encoder = LabelEncoder()
y_train = label_encoder.fit_transform(train_labels)
y_test = label_encoder.transform(test_labels)

# 6 روی داده‌های آموزشی LinearSVC آموزش مدل
svm_classifier = LinearSVC(C=1.0, max_iter=10000)
svm_classifier.fit(X_train, y_train)

# 7 پیش‌بینی برچسب‌ها روی داده‌های آزمون
predictions = svm_classifier.predict(X_test)

# 8 ارزیابی عملکرد مدل با دقت و گزارش طبقه‌بندی
accuracy = accuracy_score(y_test, predictions)
classification_rep = classification_report(y_test, predictions, target_names=newsgroups.target_names)

print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_rep)
```

Accuracy: 0.62
Classification Report:

	precision	recall	f1-score	support
alt.atheism	0.41	0.39	0.40	36
comp.graphics	0.57	0.62	0.59	53
comp.os.ms-windows.misc	0.57	0.61	0.59	54
comp.sys.ibm.pc.hardware	0.61	0.58	0.59	52
comp.sys.mac.hardware	0.63	0.67	0.65	46
comp.windows.x	0.72	0.71	0.72	70
misc.forsale	0.62	0.64	0.63	47
rec.autos	0.36	0.84	0.51	37
rec.motorcycles	0.84	0.67	0.75	64
rec.sport.baseball	0.63	0.60	0.62	55
rec.sport.hockey	0.74	0.77	0.76	48
sci.crypt	0.77	0.49	0.60	47
sci.electronics	0.66	0.55	0.60	67
sci.med	0.50	0.68	0.58	31
sci.space	0.81	0.62	0.71	56
soc.religion.christian	0.70	0.72	0.71	64
talk.politics.guns	0.57	0.63	0.60	52
talk.politics.mideast	0.78	0.67	0.72	54
talk.politics.misc	0.59	0.48	0.53	42
talk.religion.misc	0.23	0.20	0.21	25
accuracy			0.62	1000
macro avg	0.62	0.61	0.60	1000
weighted avg	0.64	0.62	0.62	1000

توضیحات مثال کد پایتون

در این مثال، از مدل **LinearSVC** برای طبقه‌بندی متون در مجموعه داده **Newsgroups 20** استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های متنی بارگذاری شده و به صورت DataFrame سازمان‌دهی شده‌اند.
- داده‌ها به دو بخش آموزش و آزمون تقسیم شده‌اند تا ارزیابی مدل امکان‌پذیر باشد.
- با استفاده از **TfidfVectorizer** متن‌ها به بردارهای عددی تبدیل شده‌اند که شامل unigram و bigram هستند.
- برچسب‌های متنی به صورت عددی رمزگذاری شده‌اند تا برای مدل قابل پردازش باشند.
- مدل **LinearSVC** آموزش دیده و روی داده‌های آزمون پیش‌بینی انجام داده است.
- در نهایت، دقت مدل و گزارش کامل طبقه‌بندی برای ارزیابی عملکرد نمایش داده شده‌اند.

این رویکرد برای مسائل طبقه‌بندی متنی بسیار مؤثر است و می‌تواند در کاربردهایی مانند دسته‌بندی ایمیل‌ها، تحلیل احساسات، یا فیلتر محتوای خبری مورد استفاده قرار گیرد.

توجه: دقت مدل در این مثال حدود ۶۲٪ است که به دلیل محدود بودن تعداد نمونه‌ها، تعداد ویژگی‌های TF-IDF و ساده بودن مدل LinearSVC می‌باشد. برای افزایش دقت می‌توان از تمام داده‌ها، n-gram های طولانی‌تر، مدل‌های پیشرفته‌تر مبتنی بر BERT و روش‌های متعادل‌سازی کلاس‌ها استفاده کرد.

12.1.2. Spatial-temporal data

کاوش داده‌های مکانی-زمانی (Spatial-temporal Data)

کاوش داده‌های مکانی-زمانی (Mining Spatial-temporal Data) شامل استخراج الگوها، روندها و دانش از مجموعه‌های داده‌ای است که شامل هر دو بعد مکانی (Spatial) و زمانی (Temporal) هستند. این نوع داده در حوزه‌های مختلفی مانند حمل و نقل، نظارت محیطی و خدمات مبتنی بر مکان شایع است. تکنیک‌های اعمال شده بر داده‌های مکانی-زمانی با هدف کشف روابط، پیش‌بینی رخدادها، آینده و بهبود تصمیم‌گیری در محیط‌های پویا صورت می‌گیرد.

1. ابعاد مکانی و زمانی (Spatial and Temporal Dimensions): داده‌های مکانی-زمانی اطلاعات مربوط به مکان‌های

جغرافیایی (مکانی) را با برچسب‌های زمانی یا بازه‌های زمانی (زمانی) ترکیب می‌کنند. این جنبه دوگانه امکان تحلیل چگونگی تکامل و تغییر پدیده‌ها در طول فضا و زمان را فراهم می‌کند.

2. تحلیل مسیر (Trajectory Analysis): داده‌های مسیر (Trajectory Data) که حرکت اشیاء را در طول زمان نشان

می‌دهند، یک شکل رایج از داده‌های مکانی-زمانی است. تحلیل مسیرها به درک الگوهایی مانند مسیرهای حمل و نقل، مهاجرت و گسترش بیماری‌ها کمک می‌کند.

3. تشخیص رویداد (Event Detection): کاوش داده‌های مکانی-زمانی برای تشخیص رویدادهایی که در مناطق و

دوره‌های زمانی خاصی رخ می‌دهند، مفید است. این می‌تواند شامل بلایای طبیعی، تحولات شهری، یا الگوهای ناهنجار در نظارت محیطی باشد.

4. مدل‌سازی پیش‌بینی‌کننده (Predictive Modeling): مدل‌سازی پیش‌بینی‌کننده در داده‌های مکانی-زمانی شامل

پیش‌بینی وضعیت‌ها یا رویدادهای آینده بر اساس الگوهای تاریخی است. این برای وظایفی مانند پیش‌بینی ترافیک، پیش‌بینی آب و هوا و مدیریت منابع ارزشمند است.

5. کاربردها (Applications): کاربردها حوزه‌های مختلفی از جمله برنامه‌ریزی شهری، همه‌گیرشناسی (Epidemiology)،

مدیریت حمل و نقل و علوم محیطی را در بر می‌گیرد، جایی که درک هر دو جنبه مکانی و زمانی حیاتی است.

اکنون، بیاید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای کاوش داده‌های مکانی-زمانی با استفاده از تحلیل مسیر (Trajectory Analysis) ارائه دهیم:

In [10]: *Folium آموزش: تحلیل مسیر مکانی-زمانی با شهرهای ایران و تصویرسازی با #*

```
import pandas as pd
import geopandas as gpd
from shapely.geometry import Point, LineString
import folium
from folium.plugins import MarkerCluster, TimestampedGeoJson
import numpy as np
from sklearn.cluster import DBSCAN

# 1 داده‌های مصنوعی شامل مختصات شهرهای ایران و زمان حرکت
trajectory_data = [
    {'timestamp': '2023-01-01 12:00:00', 'latitude': 35.6892, 'longitude': 51.3890}, # تهران
    {'timestamp': '2023-01-01 12:30:00', 'latitude': 29.5918, 'longitude': 50.5830}, # اهواز
    {'timestamp': '2023-01-01 13:00:00', 'latitude': 32.6546, 'longitude': 51.6680}, # اصفهان
    {'timestamp': '2023-01-01 13:30:00', 'latitude': 38.0962, 'longitude': 46.2738}, # تبریز
    {'timestamp': '2023-01-01 14:00:00', 'latitude': 29.5910, 'longitude': 52.5830}, # شیراز
    {'timestamp': '2023-01-01 14:30:00', 'latitude': 36.2605, 'longitude': 59.6168}, # مشهد
]

# 2 با datetime به هندسه نقطه‌ای و تبدیل زمان به GeoDataFrame ساخت
gdf = gpd.GeoDataFrame(
    trajectory_data,
    geometry=[Point(d['longitude'], d['latitude']) for d in trajectory_data]
)
gdf['timestamp'] = pd.to_datetime(gdf['timestamp'])

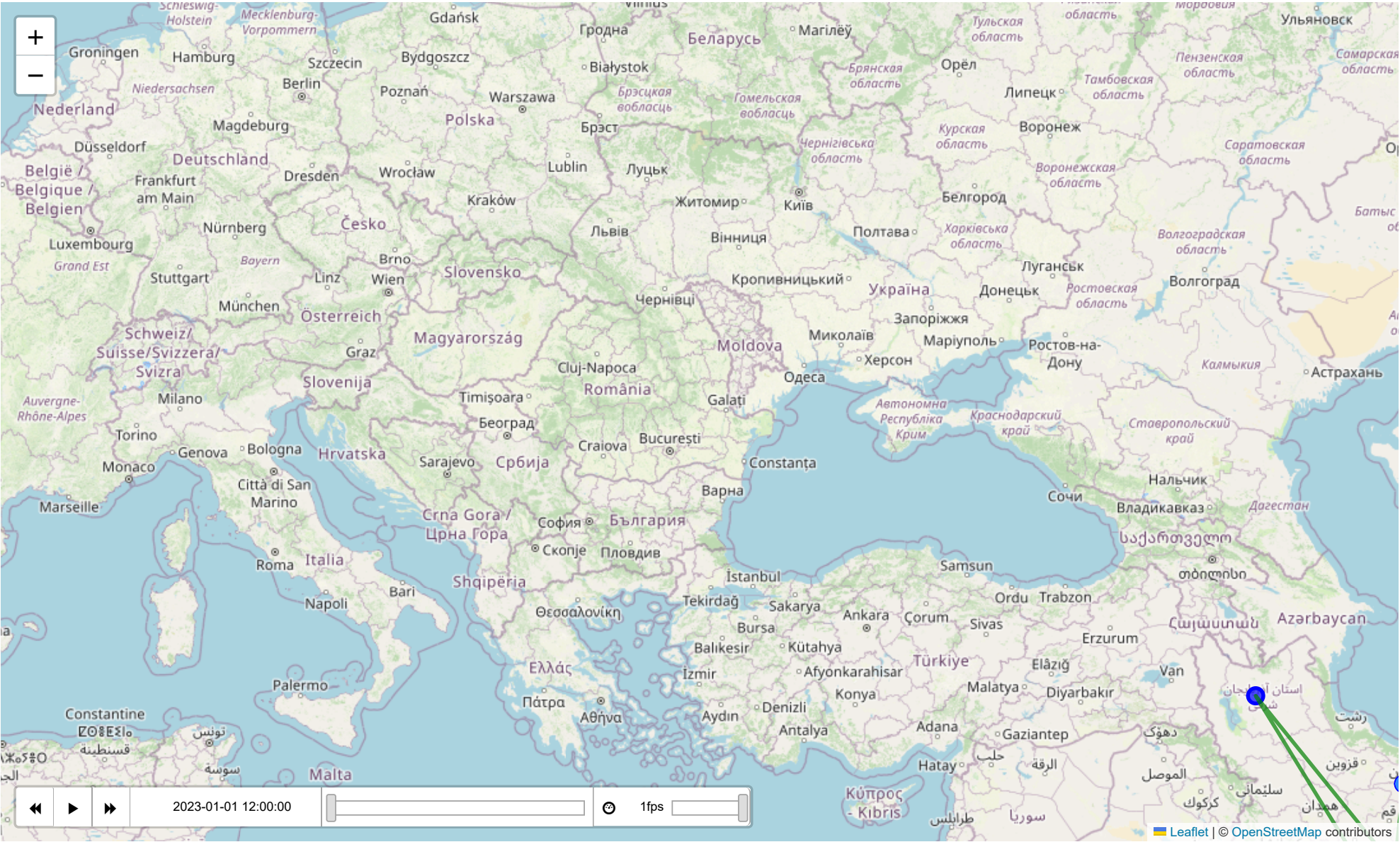
# 3 Haversine محاسبه سرعت بین نقاط با استفاده از فرمول
def haversine(lon1, lat1, lon2, lat2):
    R = 6371 # شعاع زمین بر حسب کیلومتر
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlon = lon2 - lon1
    dlat = lat2 - lat1
    a = np.sin(dlat/2)**2 + np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2
    c = 2*np.arcsin(np.sqrt(a))
    return R * c

speeds = []
for i in range(1, len(gdf)):
    dist = haversine(
        gdf.iloc[i-1]['longitude'], gdf.iloc[i-1]['latitude'],
        gdf.iloc[i]['longitude'], gdf.iloc[i]['latitude']
    )
    time_diff = (gdf.iloc[i]['timestamp'] - gdf.iloc[i-1]['timestamp']).total_seconds()/3600
    speed = dist / time_diff if time_diff > 0 else 0
    speeds.append(speed)
speeds.insert(0, 0)
gdf['speed_kmh'] = speeds

# 4 DBSCAN خوشه‌بندی مکانی با الگوریتم
coords = gdf[['longitude', 'latitude']].to_numpy()
db = DBSCAN(eps=1, min_samples=1).fit(coords)
```

```
gdf['cluster'] = db.labels_  
  
# 5 ساخت نقشه تعاملی با نمایش نقاط و خوشه‌ها  
map_center = [gdf['latitude'].mean(), gdf['longitude'].mean()]  
m = folium.Map(location=map_center, zoom_start=5)  
  
marker_cluster = MarkerCluster().add_to(m)  
for idx, row in gdf.iterrows():  
    folium.CircleMarker(  
        location=[row['latitude'], row['longitude']],  
        radius=6,  
        color='red' if row['cluster']==-1 else 'blue',  
        fill=True,  
        fill_opacity=0.7,  
        popup=f"Time: {row['timestamp']}<br>Speed: {row['speed_kmh']:.2f} km/h<br>Cluster: {row['cluster']}"  
    ).add_to(marker_cluster)  
  
# 6 ترسیم مسیر حرکت با خط سبز  
trajectory_line = LineString(gdf['geometry'])  
folium.GeoJson(  
    trajectory_line,  
    style_function=lambda x: {'color': 'green', 'weight': 3, 'opacity': 0.7}  
).add_to(m)  
  
# 7 افزودن انیمیشن زمانی با  
features = []  
for idx, row in gdf.iterrows():  
    feature = {  
        'type': 'Feature',  
        'geometry': {'type': 'Point', 'coordinates': [row['longitude'], row['latitude']]},  
        'properties': {  
            'time': row['timestamp'].strftime('%Y-%m-%dT%H:%M:%S'),  
            'popup': f"Speed: {row['speed_kmh']:.2f} km/h<br>Cluster: {row['cluster']}",  
            'icon': 'circle',  
            'iconstyle': {'fillColor': 'orange', 'fillOpacity': 0.6, 'stroke': 'true', 'radius': 5}  
        }  
    }  
    features.append(feature)  
  
TimestampedGeoJson(  
    {'type': 'FeatureCollection', 'features': features},  
    period='PT30M',  
    add_last_point=True,  
    auto_play=False,  
    loop=False,  
    max_speed=1  
).add_to(m)  
  
# 8 نمایش نقشه تعاملی  
m
```

Out[10]:



📖 تحلیل مسیر مکانی-زمانی با شهرهای ایران و Folium

در این پروژه، یک مسیر فرضی بین چند شهر ایران طراحی شده و با استفاده از الگوریتم‌های مکانی و ابزارهای تصویری، تحلیل‌های زیر انجام شده‌اند:

- داده‌های مکانی-زمانی شامل مختصات جغرافیایی و زمان حرکت در قالب `GeoDataFrame` ساخته شده‌اند.
- با استفاده از فرمول **Haversine**، فاصله بین نقاط محاسبه شده و سرعت حرکت بین آن‌ها استخراج شده است.
- الگوریتم **DBSCAN** برای خوشه‌بندی مکانی اجرا شده تا نقاط پرت یا تجمعی شناسایی شوند.
- نقشه تعاملی با **Folium** ساخته شده و نقاط با رنگ‌بندی خوشه‌ها و مسیر حرکت با خط سبز نمایش داده شده‌اند.
- انیمیشن زمانی با استفاده از `TimestampedGeoJson` اضافه شده تا تغییرات مکانی در طول زمان قابل مشاهده باشند.

این رویکرد برای تحلیل مسیرهای حمل‌ونقل، پایش حرکت کاربران، یا شناسایی رویدادهای مکانی-زمانی در ایران بسیار کاربردی است. می‌توان آن را با داده‌های واقعی GPS یا سامانه‌های شهری ترکیب کرد تا در پروژه‌های هوشمند، لجستیکی یا امنیتی استفاده شود.

12.1.3. Graph and networks

کاوش داده‌های گراف و شبکه (Mining graph and network data)

کاوش داده‌های گراف و شبکه (Mining graph and network data) شامل تحلیل موجودیت‌های به‌هم‌پیوسته و روابط بین آن‌ها است. این نوع داده به عنوان گراف نمایش داده می‌شود، جایی که گره‌ها (nodes) موجودیت‌ها و یال‌ها (edges) اتصالات یا تعاملات را نشان می‌دهند. تکنیک‌های کاوش داده‌های گراف برای درک روابط پیچیده، شناسایی الگوها و استخراج بینش‌های معنادار از سیستم‌های به‌هم‌پیوسته ضروری هستند.

نکات کلیدی درباره کاوش داده‌های گراف و شبکه:

- **نمایش گراف (Graph Representation):** گراف‌ها از گره‌ها (vertices) و یال‌هایی که این گره‌ها را به هم متصل می‌کنند، تشکیل شده‌اند. این نمایش برای مدل‌سازی روابط در حوزه‌های متنوعی مانند شبکه‌های اجتماعی، سیستم‌های حمل و نقل، تعاملات بیولوژیکی و موارد دیگر استفاده می‌شود.
- **تشخیص جامعه (Community Detection):** الگوریتم‌های تشخیص جامعه، گروه‌هایی از گره‌ها را شناسایی می‌کنند که در داخل خود به شدت به هم متصل هستند، اما به بقیه گراف به طور پراکنده متصل می‌باشند. این امر به درک ساختار ماژولار شبکه‌های پیچیده کمک می‌کند.
- **معیارهای مرکزیت (Centrality Measures):** معیارهای مرکزیت، مانند مرکزیت درجه (degree centrality) و مرکزیت بینابینی (betweenness centrality)، اهمیت گره‌ها را در یک شبکه کمی می‌کنند. این معیارها به شناسایی موجودیت‌های تأثیرگذار و اتصال‌دهنده‌های کلیدی کمک می‌کنند.
- **تشخیص ناهنجاری (Anomaly Detection):** کاوش داده‌های گراف برای تشخیص ناهنجاری‌ها (anomalies) یا داده‌های پرت (outliers) در سیستم‌های به‌هم‌پیوسته ارزشمند است. تغییرات ناگهانی در ساختار شبکه یا اتصالات غیرمنتظره می‌تواند نشان‌دهنده ناهنجاری باشد.
- **سیستم‌های توصیه‌گر (Recommendation Systems):** سیستم‌های توصیه‌گر مبتنی بر گراف از روابط بین موجودیت‌ها برای ارائه توصیه‌های شخصی‌سازی‌شده بهره می‌برند. این در رسانه‌های اجتماعی، تجارت الکترونیکی (e-commerce) و پلتفرم‌های محتوا رایج است.

اکنون، بیا یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای کاوش داده‌های گراف و شبکه با استفاده از تشخیص جامعه ارائه دهیم:

```
In [12]: # آموزش: تشخیص اجتماع در گراف شبکه اجتماعی با استفاده از مدولاریتی

import networkx as nx
import matplotlib.pyplot as plt
from networkx.algorithms.community import greedy_modularity_communities

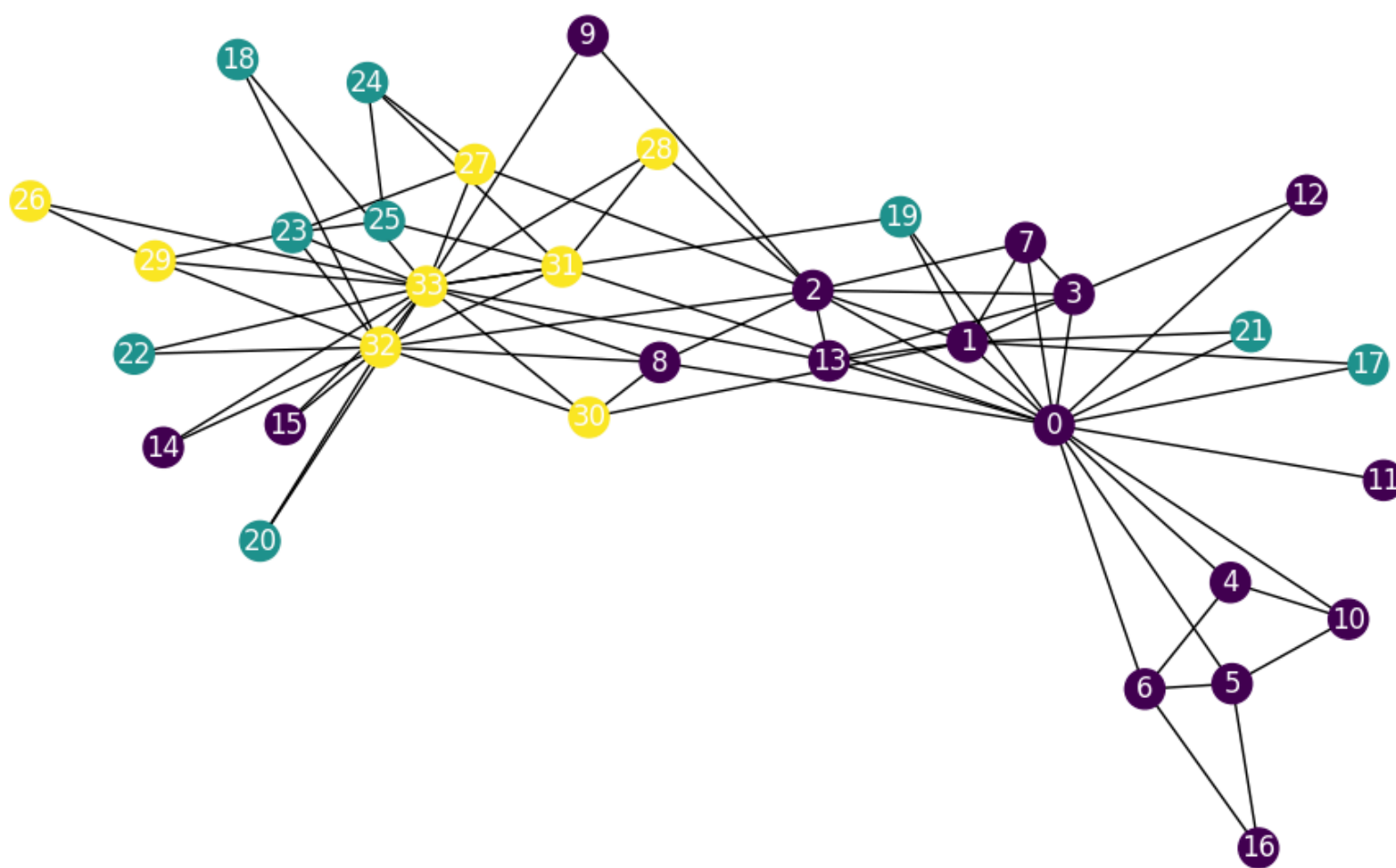
# 1 تولید گراف مصنوعی شبکه اجتماعی (باشگاه کاراته)
G = nx.karate_club_graph()

# 2 (greedy) تشخیص اجتماعات با الگوریتم مدولاریتی
communities = greedy_modularity_communities(G)

# 3 تعیین رنگ گره‌ها بر اساس اجتماع مربوطه
node_colors = []
for i, community in enumerate(communities):
    node_colors.extend([i] * len(community))

# 4 رسم گراف با رنگ‌بندی گره‌ها بر اساس اجتماع
plt.figure(figsize=(10, 6))
nx.draw(
    G,
    with_labels=True,
    node_color=node_colors,
    cmap=plt.cm.viridis,
    font_color='white'
)
plt.title('Social Network Graph with Community Detection')
plt.show()
```

Social Network Graph with Community Detection



تشخیص اجتماع در گراف شبکه اجتماعی با مدولاریتی

در این مثال، از گراف معروف باشگاه کاراته برای نمایش ساختار شبکه اجتماعی استفاده شده و با الگوریتم مدولاریتی، اجتماعات درون شبکه شناسایی شده‌اند. مراحل انجام‌شده به شرح زیر است:

- ابتدا گراف شبکه اجتماعی با استفاده از تابع `karate_club_graph` ساخته شده است.
- الگوریتم `greedy_modularity_communities` برای تشخیص گروه‌های مرتبط در شبکه اجرا شده است.
- هر اجتماع با رنگ متفاوت نمایش داده شده تا ساختار گروهی شبکه به صورت بصری قابل مشاهده باشد.
- نقشه گراف با برچسب گره‌ها و رنگ‌بندی اجتماعات رسم شده است.

تشخیص اجتماع در گراف‌ها یکی از ابزارهای کلیدی در تحلیل شبکه‌های اجتماعی، ارتباطات سازمانی، و ساختارهای پیچیده است. این روش می‌تواند برای تحلیل رفتار کاربران، کشف گروه‌های پنهان، یا بهینه‌سازی ساختار شبکه‌ها کاربرد داشته باشد.

Section 12.2

12.2.1. Data mining for sentiment and opinion

کاوش داده‌ها برای احساسات و نظرات (Data Mining for Sentiment and Opinion)

کاوش داده‌ها برای احساسات و نظرات شامل تحلیل داده‌های متنی برای کشف احساسات، عواطف و نظرات بیان‌شده توسط افراد یا گروه‌ها است. این کاربرد به طور گسترده‌ای در رسانه‌های اجتماعی، نقدهای مشتریان و سایر پلتفرم‌ها که درک احساسات عمومی در آن‌ها حیاتی است، استفاده می‌شود. تکنیک‌هایی مانند پردازش زبان طبیعی (Natural Language Processing - NLP)، تحلیل احساسات (Sentiment Analysis) و یادگیری ماشین (Machine Learning) برای استخراج بینش‌های ارزشمند از داده‌های متنی بدون ساختار به کار می‌روند.

نکات کلیدی درباره کاوش داده‌ها برای احساسات و نظرات:

- تحلیل متنی (Textual Analysis):** تکنیک‌های کاوش داده‌ها برای تحلیل حجم زیادی از داده‌های متنی، مانند پست‌های رسانه‌های اجتماعی، نقدهای محصول و نظرات، اعمال می‌شوند. هدف، استخراج احساسات، عواطف و نظرات بیان‌شده توسط کاربران است.
- تکنیک‌های تحلیل احساسات (Sentiment Analysis Techniques):** تحلیل احساسات شامل طبقه‌بندی متن به دسته‌هایی مانند مثبت، منفی یا خنثی است. تکنیک‌ها از رویکردهای مبتنی بر قاعده تا مدل‌های یادگیری ماشین، از جمله پردازش زبان طبیعی (NLP) و روش‌های یادگیری عمیق (Deep Learning)، متغیر هستند.
- نظارت بر برند (Brand Monitoring):** کسب‌وکارها از تحلیل احساسات برای نظارت و درک اینکه برند آن‌ها چگونه توسط مشتریان درک می‌شود، استفاده می‌کنند. تحلیل ذکرها در رسانه‌های اجتماعی و نقدها به ارزیابی افکار عمومی و اتخاذ تصمیمات آگاهانه کمک می‌کند.
- بازخورد مشتریان (Customer Feedback):** تحلیل احساسات بر روی نقدها و بازخوردهای مشتریان اعمال می‌شود تا زمینه‌های بهبود، نقاط درد مشترک و رضایت کلی مشتری شناسایی شود. این برای ارتقاء محصولات و خدمات ارزشمند است.
- تحلیل‌گرهای رسانه‌های اجتماعی (Social Media Analytics):** پلتفرم‌های رسانه‌های اجتماعی حجم عظیمی از داده‌های متنی تولید می‌کنند. کاوش داده‌ها برای احساسات به کسب‌وکارها، بازاریابان و محققان اجازه می‌دهد تا روندها را پیگیری کنند، احساسات عمومی را درک کنند و به طور مؤثر پاسخ دهند.

یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای کاوش داده‌ها برای احساسات و نظرات با استفاده از یک مدل تحلیل احساسات:

In [14]: `matplotlib` و `TextBlob` آموزش: تحلیل احساسات متنی با استفاده از `#`

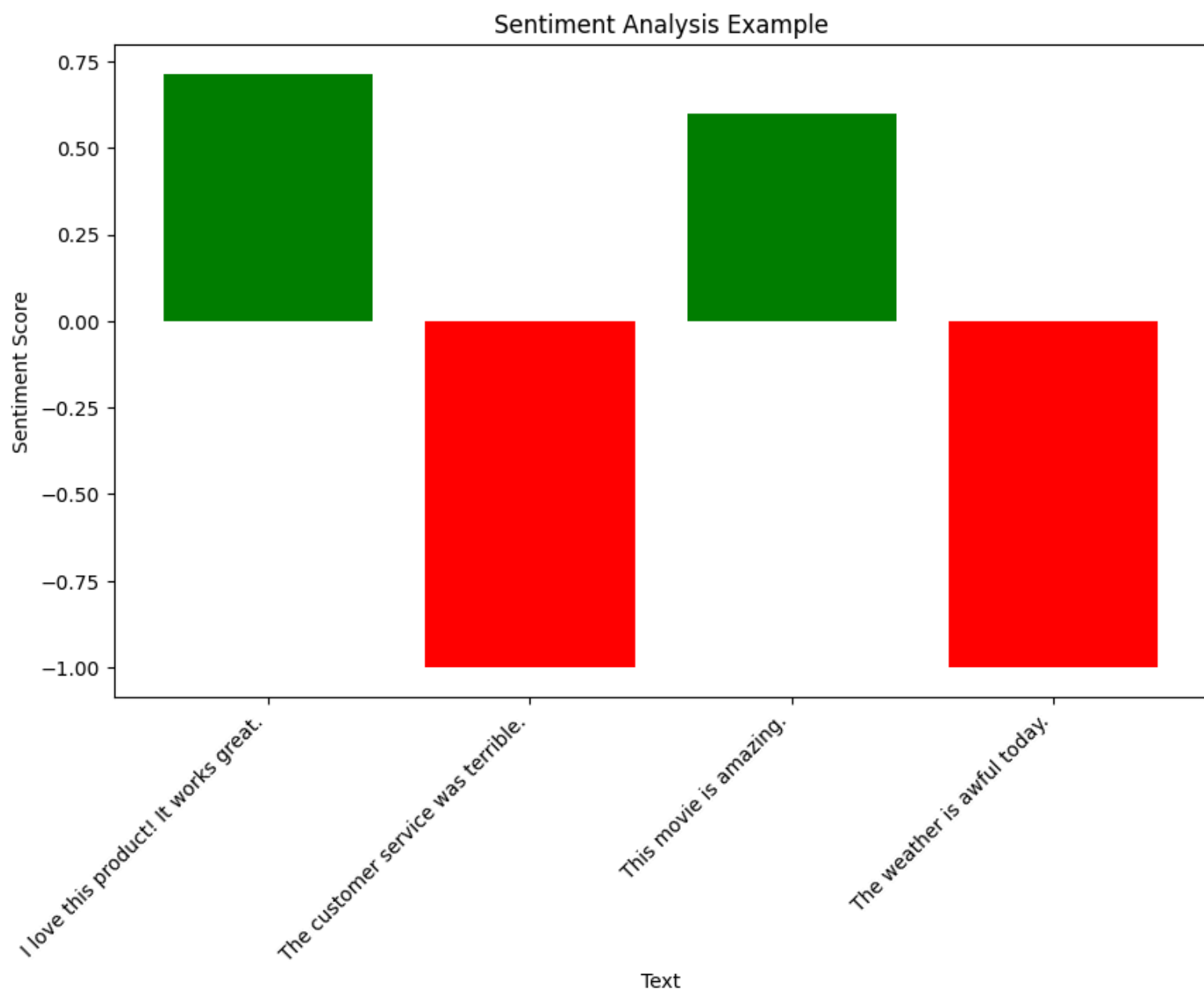
```
from textblob import TextBlob
import matplotlib.pyplot as plt
import pandas as pd
```

```
# 1 داده‌های نمونه شامل چند جمله با بار احساسی مختلف
data = {'Text': [
    'I love this product! It works great.',      # مثبت
    'The customer service was terrible.',        # منفی
    'This movie is amazing.',                   # مثبت
    'The weather is awful today.'               # منفی
]}

df = pd.DataFrame(data)

# 2 TextBlob اعمال تحلیل احساسات با استفاده از
df['Sentiment'] = df['Text'].apply(lambda x: TextBlob(x).sentiment.polarity)

# 3 رسم نمودار میله‌ای برای نمایش نمرات احساسات
plt.figure(figsize=(10, 6))
plt.bar(
    df.index,
    df['Sentiment'],
    color=df['Sentiment'].apply(lambda x: 'green' if x > 0 else 'red')
)
plt.title('Sentiment Analysis Example')
plt.xlabel('Text')
plt.ylabel('Sentiment Score')
plt.xticks(df.index, df['Text'], rotation=45, ha='right')
plt.show()
```



تحلیل احساسات متنی با TextBlob و نمایش بصری

در این مثال، از کتابخانه **TextBlob** برای تحلیل احساسات جملات انگلیسی استفاده شده و نتایج به صورت نمودار میله‌ای نمایش داده شده‌اند. مراحل انجام‌شده به شرح زیر است:

- ابتدا چند جمله‌ی نمونه با بار احساسی مختلف (مثبت و منفی) تعریف شده‌اند.
- با استفاده از تابع `sentiment.polarity`، نمره‌ی احساسات هر جمله بین -1 تا +1 محاسبه شده است.
- جملات با نمره مثبت به رنگ سبز و جملات با نمره منفی به رنگ قرمز در نمودار نمایش داده شده‌اند.
- نمودار نهایی با چرخش برچسب‌ها و اندازه مناسب برای خوانایی بهتر تنظیم شده است.

این روش برای تحلیل سریع احساسات در نظرات کاربران، بازخورد مشتریان، یا بررسی محتوای متنی بسیار مفید است. می‌توان آن را با داده‌های واقعی ترکیب کرد تا در سیستم‌های هوشمند پاسخ‌گویی، داشبوردهای تحلیل بازخورد، یا پایش شبکه‌های اجتماعی استفاده شود.

کشف حقیقت (Truth discovery) و شناسایی اطلاعات غلط (misinformation identification)

کشف حقیقت (Truth discovery) شامل فرآیند تعیین دقیق‌ترین و قابل‌اعتمادترین اطلاعات از منابع متضاد یا نامشخص است. این امر به ویژه در سناریوهایی که چندین منبع اطلاعات متناقض ارائه می‌دهند، مرتبط است و هدف، شناسایی داده‌های واقعی یا صحیح است. شناسایی اطلاعات غلط یک وظیفه مرتبط است که هدف آن تشخیص اطلاعات نادرست یا گمراه‌کننده در مجموعه‌های داده است. تکنیک‌های کاوش داده‌ها برای ارزیابی اعتبار منابع و کشف اطلاعات دقیق به کار گرفته می‌شوند.

نکات کلیدی درباره کشف حقیقت و شناسایی اطلاعات غلط:

- **اطلاعات متضاد (Conflicting Information):** در مجموعه‌های داده‌ای با اطلاعات متضاد از منابع متعدد، تعیین داده‌های واقعی یا دقیق می‌تواند چالش‌برانگیز باشد. روش‌های کشف حقیقت با هدف آشتی دادن این تضادها و شناسایی قابل‌اعتمادترین اطلاعات صورت می‌گیرند.
- **اعتبار منبع (Source Credibility):** ارزیابی اعتبار منابع اطلاعات حیاتی است. کشف حقیقت شامل اختصاص وزن یا امتیاز به منابع مختلف بر اساس دقت تاریخی، قابلیت اطمینان یا تخصص آن‌ها در یک حوزه مشخص است.
- **تحلیل سازگاری (Consistency Analysis):** تحلیل سازگاری یک رویکرد رایج در کشف حقیقت است. این شامل ارزیابی سازگاری اطلاعات در میان چندین منبع است. اطلاعات ناسازگار ممکن است نشان‌دهنده خطاها یا اطلاعات غلط عمدی باشند.
- **تشخیص اطلاعات غلط (Misinformation Detection):** شناسایی اطلاعات غلط بر روی تشخیص اطلاعات نادرست یا گمراه‌کننده‌ای که به طور عمدی برای فریب‌دادن منتشر شده‌اند، تمرکز دارد. تکنیک‌های کاوش داده‌ها، از جمله پردازش زبان طبیعی و یادگیری ماشین، برای شناسایی الگوهای مرتبط با اطلاعات غلط به کار می‌روند.
- **کاربردها (Applications):** کشف حقیقت و شناسایی اطلاعات غلط در حوزه‌های مختلفی از جمله روزنامه‌نگاری، رسانه‌های اجتماعی و تصمیم‌گیری مبتنی بر داده کاربرد دارند، جایی که قابلیت اطمینان اطلاعات حیاتی است.

اکنون، بیا یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای کشف حقیقت و شناسایی اطلاعات غلط با استفاده از یک تحلیل سازگاری پایه ارائه دهیم:

In [17]: آموزش: کشف حقیقت و شناسایی اطلاعات غلط با استفاده از اعتبار منابع و تحلیل فراوانی

```
import pandas as pd
from collections import defaultdict
import numpy as np

# تولید داده‌های نمونه شامل منابع مختلف و جملات متناقض درباره تغییرات اقلیمی
data = [
    {'source': 'News_A', 'statement': 'Climate change is real.'},
    {'source': 'News_B', 'statement': 'Climate change is a hoax.'},
    {'source': 'Research_C', 'statement': 'Climate change is real.'},
    {'source': 'Blog_D', 'statement': 'Climate change is exaggerated.'},
    {'source': 'News_E', 'statement': 'Climate change is real.'},
]
df = pd.DataFrame(data)

# تعریف اعتبار اولیه برای هر منبع (به صورت پیش‌فرض 0.8)
source_weights = {src: 0.8 for src in df['source'].unique()}

# محاسبه امتیاز هر جمله بر اساس مجموع اعتبار منابعی که آن را بیان کرده‌اند
statement_scores = defaultdict(float)
for idx, row in df.iterrows():
    statement_scores[row['statement']] += source_weights[row['source']]
```

```
"نرمال‌سازی امتیازها و انتخاب جمله‌ای با بیشترین امتیاز به‌عنوان "حقیقت" 4
total_score = sum(statement_scores.values())
for stmt in statement_scores:
    statement_scores[stmt] /= total_score

truth_statement = max(statement_scores, key=statement_scores.get)

# شناسایی اطلاعات غلط: جملاتی که با حقیقت نهایی همخوانی ندارند 5
df['is_misinformation'] = df['statement'].apply(lambda x: x != truth_statement)

# نمایش نتایج نهایی 6
print("♦ حقیقت کشف شده (Truth Statement):", truth_statement)
print("\n♦ امتیاز صحت جملات (Normalized Scores):")
for stmt, score in statement_scores.items():
    print(f"    {stmt}: {score:.2f}")

print("\n♦ جدول منابع و شناسایی اطلاعات غلط")
print(df)
```

♦ حقیقت کشف شده (Truth Statement): Climate change is real.

♦ امتیاز صحت جملات (Normalized Scores):
Climate change is real.: 0.60
Climate change is a hoax.: 0.20
Climate change is exaggerated.: 0.20

♦ جدول منابع و شناسایی اطلاعات غلط:

	source	statement	is_misinformation
0	News_A	Climate change is real.	False
1	News_B	Climate change is a hoax.	True
2	Research_C	Climate change is real.	False
3	Blog_D	Climate change is exaggerated.	True
4	News_E	Climate change is real.	False

کشف حقیقت و شناسایی اطلاعات غلط با تحلیل منابع

در این مثال، مجموعه‌ای از جملات متناقض درباره تغییرات اقلیمی از منابع مختلف بررسی شده‌اند. هدف این تحلیل، کشف جمله‌ای است که بیشترین اعتبار را دارد و شناسایی جملاتی که با آن همخوانی ندارند. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌هایی شامل منابع خبری، پژوهشی و بلاگ‌ها با جملات متناقض تولید شده‌اند.
- برای هر منبع، اعتبار اولیه تعریف شده و امتیاز هر جمله بر اساس منابعی که آن را بیان کرده‌اند محاسبه شده است.
- امتیازها نرمال‌سازی شده‌اند تا سهم نسبی هر جمله مشخص شود.
- جمله‌ای با بیشترین امتیاز به‌عنوان "حقیقت نهایی" انتخاب شده است.
- جملاتی که با حقیقت همخوانی ندارند به‌عنوان اطلاعات غلط علامت‌گذاری شده‌اند.

این رویکرد برای تحلیل تضادهای اطلاعاتی، ارزیابی اعتبار منابع، و شناسایی محتوای گمراه‌کننده در حوزه‌هایی مانند رسانه، سیاست، یا سلامت عمومی بسیار کاربردی است. می‌توان آن را با مدل‌های زبانی پیشرفته یا شبکه‌های دانش ترکیب کرد تا دقت تحلیل افزایش یابد.

12.2.3. Information and disease propagation

کاربردهای کاوش داده‌ها در انتشار اطلاعات و بیماری (Information and Disease Propagation)

کاربردهای کاوش داده‌ها در انتشار اطلاعات و بیماری شامل تحلیل چگونگی گسترش اطلاعات و بیماری‌ها در شبکه‌ها است. درک پویایی انتشار اطلاعات در شبکه‌های اجتماعی و الگوهای انتقال بیماری‌ها برای اتخاذ تصمیمات آگاهانه در زمینه‌هایی مانند سلامت عمومی، مدیریت بحران و تحلیل نفوذ اجتماعی حیاتی است.

نکات کلیدی درباره انتشار اطلاعات و بیماری:

- **پویایی شبکه (Network Dynamics):** اطلاعات و بیماری‌ها اغلب از طریق شبکه‌های به هم پیوسته گسترش می‌یابند. تحلیل پویایی این شبکه‌ها بینش‌هایی درباره عوامل مؤثر بر الگوهای انتشار فراهم می‌کند.
- **تحلیل نفوذ اجتماعی (Social Influence Analysis):** تکنیک‌های کاوش داده‌ها برای مطالعه نفوذ اجتماعی در داخل شبکه‌ها به کار می‌روند. شناسایی گره‌های تأثیرگذار و درک چگونگی گسترش اطلاعات از طریق اتصالات اجتماعی به طراحی استراتژی‌های مؤثر برای انتشار اطلاعات کمک می‌کند.
- **مدل‌سازی اپیدمیولوژیک (Epidemiological Modeling):** در زمینه انتشار بیماری، کاوش داده‌ها برای توسعه مدل‌های اپیدمیولوژیک (Epidemiological Models) که گسترش بیماری‌ها را شبیه‌سازی می‌کنند، استفاده می‌شود. این مدل‌ها عواملی مانند الگوهای تماس، نرخ انتقال و پویایی جمعیت را در نظر می‌گیرند.
- **تشخیص زودهنگام و مداخله (Early Detection and Intervention):** کاوش داده‌ها امکان تشخیص زودهنگام روندهای نوظهور در انتشار اطلاعات یا شیوع بیماری را فراهم می‌سازد. مداخله به موقع بر اساس بینش‌های مبتنی بر داده برای مدیریت و کاهش تأثیر این پدیده‌ها حیاتی است.
- **پشتیبانی از تصمیم‌گیری در سلامت عمومی (Public Health Decision Support):** تصمیم‌گیرندگان در حوزه سلامت عمومی از نتایج کاوش داده‌ها برای اطلاع‌رسانی به استراتژی‌های کمپین‌های واکسیناسیون، برنامه‌های آگاهی‌رسانی عمومی و تخصیص منابع در پاسخ به شیوع بیماری‌ها استفاده می‌کنند.

یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای انتشار اطلاعات و بیماری با استفاده از یک مدل اپیدمیولوژیک (Epidemiological Model):

In [20]: *SIR آموزش: شبیه‌سازی انتشار اطلاعات یا بیماری در شبکه اجتماعی با مدل*

```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
from random import sample

# 1 Erdős-Rényi تولید شبکه اجتماعی مصنوعی با مدل
num_nodes = 50
edge_prob = 0.1
G = nx.erdos_renyi_graph(num_nodes, edge_prob)

# 2 انتخاب تصادفی گره‌های اولیه آلوده یا منتشرکننده اطلاعات
initial_infected = sample(list(G.nodes), 3)

# 3 SIR تعریف تابع شبیه‌سازی مدل
def simulate_spread(graph, initial_infected, beta, gamma, max_steps=10):
    """
    beta: نرخ انتقال
    gamma: نرخ بهبودی
    """
    state = {node: 'S' for node in graph.nodes} # همه گره‌ها در ابتدا سالم هستند
    for node in initial_infected:
        state[node] = 'I' # گره‌های اولیه آلوده

    history = [state.copy()]

    for step in range(max_steps):
        new_state = state.copy()
        for node in graph.nodes:
            if state[node] == 'I':
                # انتقال به همسایگان سالم
                for neighbor in graph.neighbors(node):
                    if state[neighbor] == 'S':
                        if np.random.rand() < beta:
                            new_state[neighbor] = 'I'
                # احتمال بهبودی
                if np.random.rand() < gamma:
                    new_state[node] = 'R'
        state = new_state
        history.append(state.copy())
        if all(v != 'I' for v in state.values()):
            break
    return history

# 4 تعریف پارامترهای مدل
beta = 0.3 # نرخ انتقال
gamma = 0.1 # نرخ بهبودی

# 5 اجرای شبیه‌سازی و ذخیره تاریخچه وضعیت گره‌ها
history = simulate_spread(G, initial_infected, beta, gamma, max_steps=15)
```

```

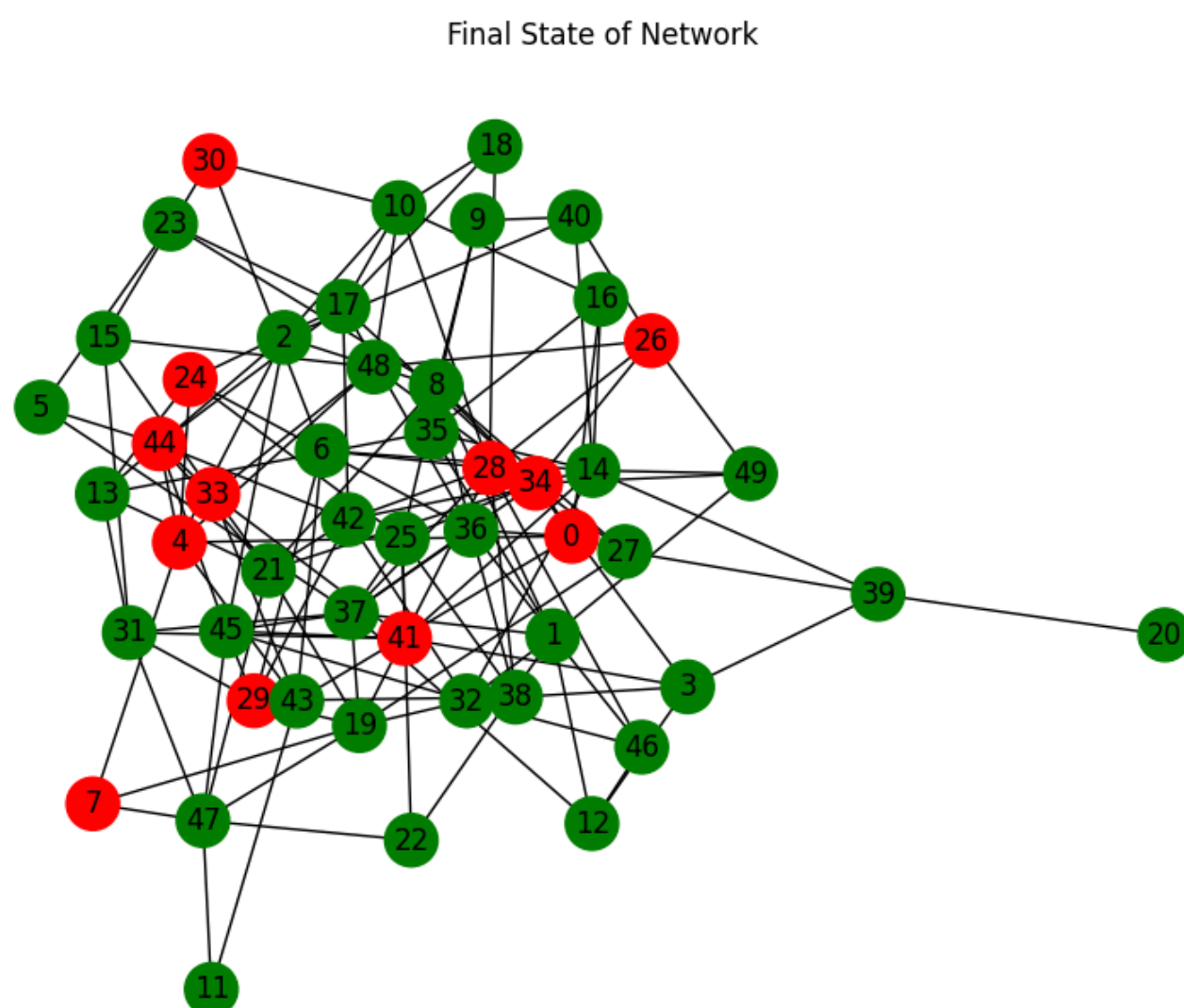
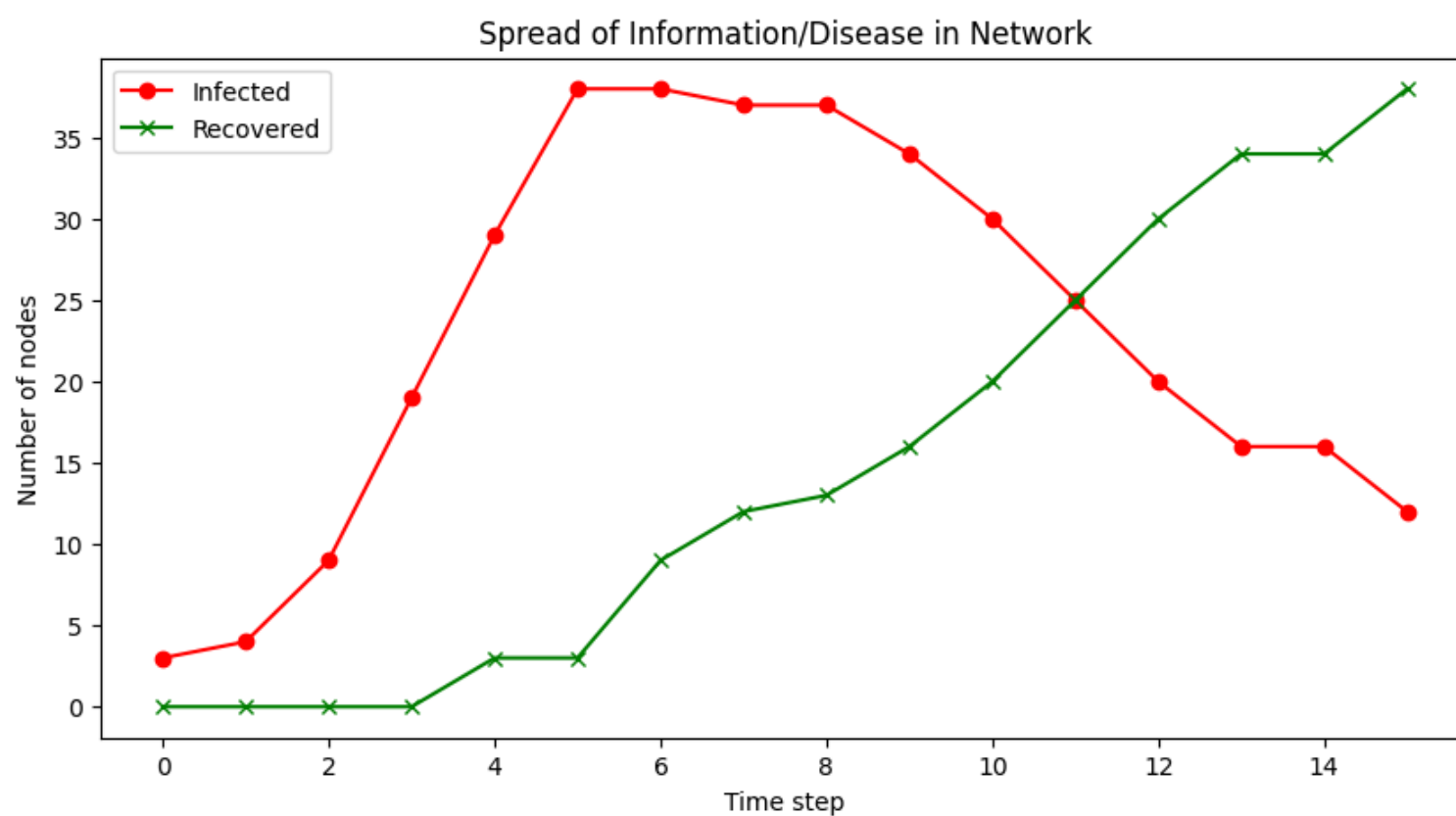
# 6 تحلیل تعداد گره‌های آلوده و بهبود یافته در هر مرحله
infected_counts = [sum(1 for v in h.values() if v=='I') for h in history]
recovered_counts = [sum(1 for v in h.values() if v=='R') for h in history]

# 7 رسم نمودار انتشار در طول زمان
plt.figure(figsize=(10,5))
plt.plot(infected_counts, label='Infected', color='red', marker='o')
plt.plot(recovered_counts, label='Recovered', color='green', marker='x')
plt.xlabel('Time step')
plt.ylabel('Number of nodes')
plt.title('Spread of Information/Disease in Network')
plt.legend()
plt.show()

# 8 نمایش وضعیت نهایی شبکه با رنگ‌بندی گره‌ها
final_state = history[-1]
node_colors = []
for node in G.nodes:
    if final_state[node]=='S':
        node_colors.append('lightblue')
    elif final_state[node]=='I':
        node_colors.append('red')
    else:
        node_colors.append('green')

plt.figure(figsize=(8,6))
nx.draw(G, with_labels=True, node_color=node_colors, node_size=500)
plt.title('Final State of Network')
plt.show()

```



شبیه‌سازی انتشار اطلاعات یا بیماری در شبکه اجتماعی با مدل SIR

در این پروژه، با استفاده از مدل SIR و شبکه اجتماعی مصنوعی، فرآیند انتشار اطلاعات یا بیماری شبیه‌سازی شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا شبکه‌ای با ۵۰ گره و احتمال اتصال ۰.۱ تولید شده است (مدل Erdős–Rényi).
 - سه گره به‌صورت تصادفی به‌عنوان گره‌های اولیه آلوده یا منتشرکننده اطلاعات انتخاب شده‌اند.
 - با استفاده از مدل SIR، وضعیت گره‌ها در هر مرحله به‌صورت S (سالم)، I (آلوده)، و R (بهبود یافته) به‌روزرسانی شده‌اند.
 - نمودار تعداد گره‌های آلوده و بهبود یافته در طول زمان رسم شده تا روند انتشار قابل مشاهده باشد.
 - در پایان، وضعیت نهایی شبکه با رنگ‌بندی گره‌ها نمایش داده شده است.
- این شبیه‌سازی برای تحلیل نفوذ اجتماعی، مدل‌سازی اپیدمی‌ها، یا بررسی انتشار شایعات و اطلاعات در شبکه‌های انسانی بسیار کاربردی است. می‌توان آن را با داده‌های واقعی، گراف‌های پیچیده‌تر، یا مدل‌های پیشرفته‌تر مانند SEIR یا SIS ترکیب کرد تا دقت تحلیل افزایش یابد.

12.2.4. Productivity and team science

کاربردهای کاوش داده‌ها در بهره‌وری و علم تیم (Productivity and team science)

کاربردهای کاوش داده‌ها در بهره‌وری و علم تیم شامل تحلیل الگوهای همکاری و عوامل بهره‌وری در داخل تیم‌ها است. درک اینکه تیم‌ها چگونه با هم کار می‌کنند، شناسایی مشارکت‌کنندگان کلیدی و بهینه‌سازی فرآیندهای همکاری برای ارتقاء بهره‌وری و تقویت نوآوری در تلاش‌های علمی ضروری است.

نکات کلیدی درباره بهره‌وری و علم تیم:

- الگوهای همکاری (Collaboration Patterns):** تکنیک‌های کاوش داده‌ها برای مطالعه الگوهای همکاری در داخل تیم‌ها به کار می‌روند. این شامل تحلیل شبکه‌های نویسندگی مشترک (co-authorship networks)، الگوهای ارتباطی و منابع مشترک برای درک چگونگی همکاری اعضای تیم است.
- معیارهای بهره‌وری (Productivity Metrics):** معیارهای بهره‌وری، مانند تعداد نشریات منتشر شده، نرخ تکمیل پروژه و سطوح مشارکت، با استفاده از روش‌های کاوش داده‌ها ارزیابی می‌شوند. این معیارها به اندازه‌گیری تأثیر و کارایی اعضای تیم و تیم‌ها به طور کلی کمک می‌کنند.
- شناسایی مشارکت‌کنندگان کلیدی (Identification of Key Contributors):** کاوش داده‌ها امکان شناسایی مشارکت‌کنندگان کلیدی در داخل تیم‌ها را فراهم می‌سازد. تحلیل مشارکت‌های فردی، تخصص و الگوهای همکاری به شناخت اعضای تیمی که به طور قابل توجهی بر بهره‌وری و موفقیت تیم تأثیر می‌گذارند، کمک می‌کند.
- بهینه‌سازی ترکیب تیم (Team Composition Optimization):** درک ویژگی‌های تیم‌های موفق برای بهینه‌سازی ترکیب تیم حیاتی است. کاوش داده‌ها می‌تواند عواملی مانند تنوع، توزیع مهارت و سبک‌های ارتباطی را که به همکاری مؤثر و بهره‌وری کمک می‌کنند، آشکار سازد.
- نوآوری و پویایی تیم (Innovation and Team Dynamics):** تحلیل داده‌ها درباره علم تیم به محققان و تصمیم‌گیرندگان اجازه می‌دهد تا بینش‌هایی در مورد پویایی نوآوری کسب کنند. این شامل شناسایی عواملی است که

همکاری بین‌رشته‌ای موفق و نوآوری در داخل تیم‌ها را هدایت می‌کنند.

یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای تحلیل الگوهای همکاری در داخل تیم‌ها:

In [24]: *آموزش: تحلیل بهره‌وری و نقش‌های کلیدی در شبکه همکاری علمی با استفاده از معیارهای گراف*

```
import networkx as nx
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# 1 داده‌های نمونه شامل نویسندگان و تعداد همکاری‌های مشترک
data = {
    'Author1': ['Alice', 'Bob', 'Alice', 'Charlie', 'David', 'Alice', 'Bob', 'Charlie'],
    'Author2': ['Bob', 'Alice', 'Charlie', 'David', 'Alice', 'Charlie', 'Charlie', 'David'],
    'Publications': [3, 2, 1, 4, 2, 1, 2, 3]
}
df = pd.DataFrame(data)

# 2 ساخت گراف همکاری با وزن لبه‌ها بر اساس تعداد پروژه‌ها
G = nx.from_pandas_edgelist(df, 'Author1', 'Author2', edge_attr='Publications', create_using=nx.Graph())

# 3 محاسبه معیارهای گرافی برای تحلیل نقش افراد
degree centrality = nx.degree_centrality(G) # میزان مشارکت در ارتباطات
betweenness centrality = nx.betweenness_centrality(G) # نقش واسطه‌گری در تیم
productivity_score = {
    node: sum([d['Publications'] for u, v, d in G.edges(node, data=True)])
    for node in G.nodes()
} # مجموع همکاری‌ها به‌عنوان شاخص بهره‌وری

# 4 نمایش اطلاعات گره‌ها به‌صورت عددی
for node in G.nodes():
    print(f"{node}: Degree Centrality={degree_centrality[node]:.2f}, "
          f"Betweenness={betweenness_centrality[node]:.2f}, "
          f"Productivity={productivity_score[node]}")

# 5 colormap نرمال‌سازی شده با رنگ گره‌ها بر اساس بهره‌وری
node_colors = [
    plt.cm.viridis(productivity_score[node] / max(productivity_score.values()))
    for node in G.nodes()
]

# 6 اندازه گره‌ها بر اساس نقش واسطه‌گری (بینابینی)
node_sizes = [
    500 + betweenness_centrality[node] * 3000
    for node in G.nodes()
]

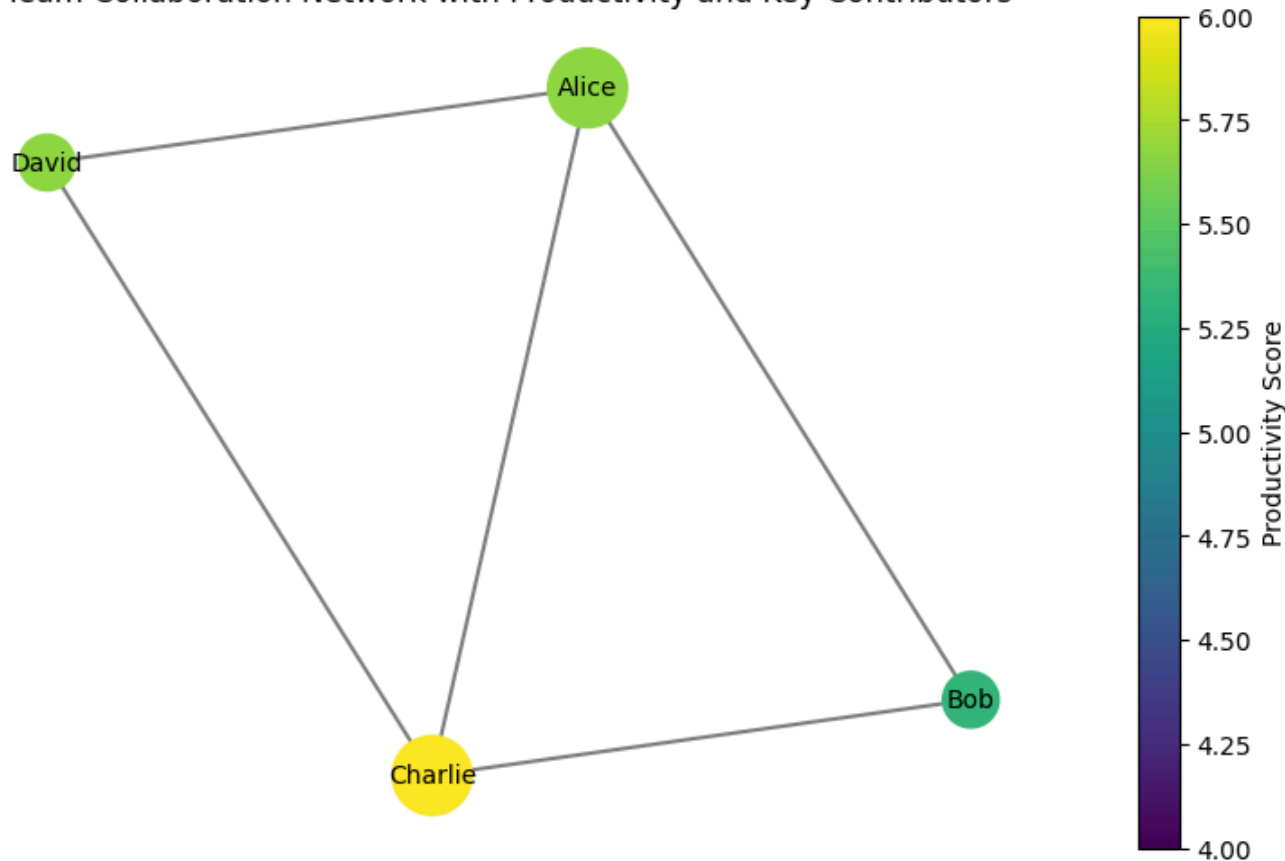
# 7 رسم گراف همکاری با رنگ و اندازه متناسب با نقش و بهره‌وری
fig, ax = plt.subplots(figsize=(10, 6))
pos = nx.spring_layout(G, seed=42)
nx.draw(
    G, pos, with_labels=True,
    node_color=node_colors,
    node_size=node_sizes,
    edge_color='gray', width=1.5,
    font_size=10, ax=ax
)

# برای نمایش بهره‌وری colorbar افزودن
sm = plt.cm.ScalarMappable(
    cmap=plt.cm.viridis,
    norm=plt.Normalize(vmin=min(productivity_score.values()), vmax=max(productivity_score.values()))
)
sm.set_array([])
cbar = fig.colorbar(sm, ax=ax)
cbar.set_label('Productivity Score')

plt.title('Team Collaboration Network with Productivity and Key Contributors')
plt.show()

# 8 شناسایی افراد کلیدی بر اساس درجه ارتباط (Degree Centrality)
key_contributors = sorted(degree_centrality, key=lambda x: degree_centrality[x], reverse=True)[:3]
print("\nKey Contributors (High Degree Centrality):", key_contributors)
```

Alice: Degree Centrality=1.00, Betweenness=0.17, Productivity=5
Bob: Degree Centrality=0.67, Betweenness=0.00, Productivity=4
Charlie: Degree Centrality=1.00, Betweenness=0.17, Productivity=6
David: Degree Centrality=0.67, Betweenness=0.00, Productivity=5



Key Contributors (High Degree Centrality): ['Alice', 'Charlie', 'Bob']

تحلیل بهره‌وری و نقش‌های کلیدی در شبکه همکاری علمی

در این پروژه، شبکه‌ای از همکاری بین نویسندگان علمی مدل‌سازی شده و با استفاده از معیارهای گرافی، نقش هر فرد در تیم تحلیل شده است. مراحل انجام‌شده به شرح زیر است:

- داده‌های همکاری شامل نویسندگان و تعداد پروژه‌های مشترک به صورت گراف بدون جهت مدل‌سازی شده‌اند.
- برای هر گره، سه معیار کلیدی محاسبه شده است: Degree Centrality برای مشارکت، Betweenness برای نقش واسطه‌گری، و Productivity برای مجموع همکاری‌ها.
- رنگ گره‌ها بر اساس بهره‌وری و اندازه آن‌ها بر اساس نقش کلیدی در ارتباطات تنظیم شده‌اند.
- نقشه گراف با layout فنی رسم شده و colorbar برای نمایش شدت بهره‌وری اضافه شده است.
- در پایان، افراد با بیشترین درجه ارتباط به عنوان اعضای کلیدی تیم شناسایی شده‌اند.

این تحلیل برای ارزیابی ساختار تیم‌های علمی، شناسایی افراد کلیدی در نوآوری، و بهینه‌سازی ترکیب تیم‌ها در پروژه‌های تحقیقاتی بسیار کاربردی است. می‌توان آن را با داده‌های واقعی از پایگاه‌های علمی مانند Scopus یا DBLP ترکیب کرد تا تحلیل‌های دقیق‌تری ارائه شود.

Section 12.3

12.3.1. Structuring unstructured data for knowledge mining: a data-driven approach

ساختمند کردن داده‌های بدون ساختار (Structuring Unstructured Data) برای کاوش دانش

ساختمند کردن داده‌های بدون ساختار یک گام حیاتی در کاوش دانش (Knowledge Mining) است، زیرا مقادیر زیادی از اطلاعات ارزشمند در قالب‌های بدون ساختار مانند متن، تصاویر و صدا وجود دارد. یک رویکرد مبتنی بر داده (Data-Driven)

شامل بهره‌گیری از تکنیک‌های کاوش داده‌ها برای استخراج ساختارها، الگوها و دانش معنادار از منابع داده‌ای بدون ساختار است. این فرآیند تبدیل داده‌های خام و بدون ساختار به قالبی مناسب برای تحلیل پیشرفته و کشف دانش را امکان‌پذیر می‌سازد.

نکات کلیدی درباره ساختمند کردن داده‌های بدون ساختار برای کاوش دانش:

- چالش‌های داده‌های بدون ساختار (Unstructured Data Challenges):** داده‌های بدون ساختار فاقد یک مدل داده‌ای از پیش تعریف‌شده هستند، که تحلیل و استخراج بینش از آن‌ها را چالش‌برانگیز می‌کند. این شامل داده‌ها به شکل اسناد متنی، تصاویر، ویدئوها و سایر رسانه‌ها است.
- ساختمندسازی مبتنی بر داده (Data-Driven Structuring):** یک رویکرد مبتنی بر داده شامل استفاده از روش‌های کاوش داده‌ها برای ساختمندسازی خودکار داده‌های بدون ساختار است. این می‌تواند شامل تکنیک‌هایی مانند پردازش زبان طبیعی (Natural Language Processing - NLP)، شناسایی تصویر (Image Recognition) و تحلیل صدا برای کشف الگوها و روابط باشد.
- استخراج ویژگی (Feature Extraction):** استخراج ویژگی یک جنبه کلیدی از ساختمندسازی داده‌های بدون ساختار است. این شامل شناسایی ویژگی‌ها یا صفات (Attributes) مرتبط از داده‌های خام است که آن‌ها را برای تحلیل با استفاده از الگوریتم‌های سنتی کاوش داده‌ها مناسب می‌سازد.
- کاوش متن و پردازش زبان طبیعی (Text Mining and NLP):** در مورد داده‌های متنی، تکنیک‌های پردازش زبان طبیعی (NLP) برای وظایفی مانند شناسایی موجودیت (Entity Recognition)، تحلیل احساسات (Sentiment Analysis) و مدل‌سازی موضوعی (Topic Modeling) اعمال می‌شوند. این تکنیک‌ها اطلاعات ساختمند را از محتوای متنی استخراج می‌کنند.
- پردازش تصویر و صدا (Image and Audio Processing):** برای داده‌های تصویری و صوتی، رویکردهای مبتنی بر داده شامل روش‌هایی مانند شناسایی تصویر (Image Recognition) و تبدیل گفتار به متن (Speech-to-Text) است. این تکنیک‌ها اطلاعات بصری و شنیداری را به نمایش‌های ساختمند تبدیل می‌کنند.
- کاربردهای دنیای واقعی (Real-World Applications):** ساختمند کردن داده‌های بدون ساختار در کاربردهایی مانند بازیابی اطلاعات (Information Retrieval)، توصیه‌گر محتوا (Content Recommendation) و تحلیل‌های پیش‌بینی‌کننده (Predictive Analytics) حیاتی است. این امر سازمان‌ها را قادر می‌سازد تا بینش‌های ارزشمندی را از منابع داده‌ای متنوع استخراج کنند.

یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای ساختمند کردن داده‌های متنی بدون ساختار با استفاده از پردازش زبان طبیعی (Natural Language Processing - NLP):

In [27]: آموزش: تبدیل داده‌های متنی غیرساخت‌یافته به داده‌های ساخت‌یافته با مدل‌سازی موضوعی و استخراج ویژگی

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.decomposition import LatentDirichletAllocation
import pandas as pd
import numpy as np

# تعریف مجموعه‌ای از جملات متنی غیرساخت‌یافته
data = {'Text': [
    'Data mining is an exciting field with vast potential.',
    'Natural language processing plays a crucial role in structuring text data.',
    'Unstructured data, when properly analyzed, reveals valuable insights.',
    'Machine learning can extract patterns from unstructured images.',
    'Speech recognition converts audio to text for analysis.'
]}

df = pd.DataFrame(data)

# بردارسازی متون با استفاده از TF-IDF
vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(df['Text'])

# مدل‌سازی موضوعی با استفاده از LDA (Latent Dirichlet Allocation)
lda = LatentDirichletAllocation(n_components=3, random_state=42)
topics = lda.fit_transform(X)

# تخصیص موضوع غالب به هر جمله و استخراج کلیدواژه‌های اصلی آن موضوع
df['Topic'] = topics.argmax(axis=1)
df['Top Keywords'] = df['Topic'].apply(
    lambda x: [vectorizer.get_feature_names_out()[i] for i in lda.components_[x].argsort()[::-4:-1]]
)
```

```
)

# استخراج ویژگی‌های ساخت‌یافته از متن (مثلاً طول جمله) 5
df['Text Length'] = df['Text'].apply(len)

# شبیه‌سازی ویژگی‌های غیرمتنی (تصویر و صوت) برای نمایش چندوجهی بودن داده‌ها 6
df['Simulated Image Feature'] = np.random.rand(len(df))
df['Simulated Audio Feature'] = np.random.rand(len(df))

# نمایش داده‌های ساخت‌یافته نهایی 7
print("Structured Data Preview:")
print(df)
```

Structured Data Preview:

	Text	Topic	\
0	Data mining is an exciting field with vast pot...	0	
1	Natural language processing plays a crucial ro...	1	
2	Unstructured data, when properly analyzed, rev...	1	
3	Machine learning can extract patterns from uns...	2	
4	Speech recognition converts audio to text for ...	2	

	Top Keywords	Text Length	Simulated Image Feature	\
0	[vast, potential, field]	53	0.866017	
1	[data, valuable, properly]	74	0.378706	
2	[data, valuable, properly]	69	0.386494	
3	[speech, recognition, audio]	63	0.606064	
4	[speech, recognition, audio]	55	0.382863	

	Simulated Audio Feature
0	0.316780
1	0.010686
2	0.557890
3	0.423901
4	0.200276

تحلیل داده‌های متنی غیرساخت‌یافته با مدل‌سازی موضوعی و استخراج



در این پروژه، مجموعه‌ای از جملات متنی غیرساخت‌یافته با استفاده از روش‌های بردارسازی و مدل‌سازی موضوعی به داده‌های ساخت‌یافته تبدیل شده‌اند. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های متنی با استفاده از **TF-IDF** بردارسازی شده‌اند تا اهمیت واژگان در هر جمله مشخص شود.
- با استفاده از الگوریتم **LDA**، موضوعات پنهان در مجموعه متون استخراج شده و به هر جمله یک موضوع غالب تخصیص داده شده است.
- برای هر موضوع، کلیدواژه‌های اصلی استخراج شده‌اند تا درک بهتری از محتوای موضوعی حاصل شود.
- ویژگی‌های عددی مانند طول جمله و ویژگی‌های شبیه‌سازی‌شده برای تصویر و صوت اضافه شده‌اند تا ساختار داده‌ها چندوجهی شود.

این رویکرد برای تبدیل داده‌های متنی به فرم قابل تحلیل در مدل‌های یادگیری ماشین، داشبوردهای تحلیلی، یا سیستم‌های توصیه‌گر بسیار کاربردی است. می‌توان آن را با داده‌های واقعی و ویژگی‌های چندرسانه‌ای ترکیب کرد تا تحلیل‌های عمیق‌تری ارائه شود.

12.3.2. Data augmentation



افزایش داده (Data augmentation)

افزایش داده (Data augmentation) یک تکنیک است که برای افزایش مصنوعی اندازه یک مجموعه داده با اعمال تغییرات مختلف بر روی داده‌های موجود استفاده می‌شود. این رویکرد به ویژه در یادگیری ماشین (machine learning) و کاوش داده‌ها (data mining) مفید است، زیرا با قرار دادن مدل در معرض تغییرات متنوع از داده اصلی، به بهبود تعمیم‌پذیری

(generalization) و عملکرد مدل کمک می‌کند. افزایش داده معمولاً در وظایفی مانند تشخیص تصویر (image recognition)،

پردازش زبان طبیعی (natural language processing) و سایر حوزه‌هایی که ممکن است مجموعه‌های داده بزرگ و برچسب‌دار

محدود باشند، به کار گرفته می‌شود.

نکات کلیدی درباره افزایش داده:

- **افزایش اندازه مجموعه داده (Increased Dataset Size):** افزایش داده شامل ایجاد نمونه‌های آموزشی جدید با

اعمال تغییرات تصادفی بر روی داده‌های موجود است. این کار به طور مؤثر اندازه مجموعه داده موجود برای آموزش

مدل را افزایش می‌دهد.

- **بهبود تعمیم‌پذیری مدل (Improving Model Generalization):** با قرار دادن مدل در معرض تغییرات متنوع داده

ورودی، افزایش داده به بهبود توانایی مدل در تعمیم به مثال‌های دیده نشده کمک می‌کند. این امر به ویژه هنگام کار

با داده‌های برچسب‌دار محدود مهم است.

- **تغییرات رایج (Common Transformations):** تکنیک‌های رایج افزایش داده شامل چرخش (rotation)، برگرداندن

(flipping)، برش (cropping)، بزرگ‌نمایی (zooming) و تغییر در روشنایی و کنتراست برای داده‌های تصویری است.

برای داده‌های متنی، افزایش داده ممکن است شامل جایگزینی مترادف، درج کلمه یا حذف باشد.

- **کاربرد در تشخیص تصویر (Application in Image Recognition):** در وظایف تشخیص تصویر، افزایش داده به طور

گسترده برای آموزش مدل‌های مقاوم در برابر تغییرات در ظاهر تصویر، مانند تغییرات در زاویه دید، شرایط نوری و

جهت‌گیری شیء استفاده می‌شود.

- **اجتناب از بیش‌برازش (Avoiding Overfitting):** افزایش داده یک تکنیک تنظیم‌کننده (regularization) است که به

جلوگیری از بیش‌برازش (overfitting) کمک می‌کند، به ویژه هنگامی که داده‌های برچسب‌دار موجود محدود هستند.

این روش تنوعی را در مجموعه آموزشی ایجاد می‌کند، و مدل را در برابر تغییرات در مجموعه آزمایشی مقاوم‌تر می‌سازد.

یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای افزایش داده در طبقه‌بندی تصویر با استفاده از کتابخانه Keras:

In [36]: *Keras در ImageDataGenerator آموزش: افزایش داده‌های تصویری با استفاده از #*

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
import numpy as np
from skimage import data

# (تصویر گربه) skimage بارگذاری تصویر نمونه داخلی از 1
image = data.chelsea() # shape: (300, 451, 3)

# نمایش تصویر اصلی 2
plt.figure(figsize=(6, 6))
plt.imshow(image)
plt.title('Original Image')
plt.axis('off')
plt.show()

# برای اعمال تبدیلات مختلف روی تصویر ImageDataGenerator تعریف شیء 3
datagen = ImageDataGenerator(
    rotation_range=40, # چرخش تصادفی تا ۴۰ درجه
    width_shift_range=0.2, # جابجایی افقی تا ۲۰٪
    height_shift_range=0.2, # جابجایی عمودی تا ۲۰٪
    shear_range=0.2, # برش زاویه‌ای
    zoom_range=0.2, # بزرگ‌نمایی تصادفی
    horizontal_flip=True, # قرینه‌سازی افقی
    fill_mode='nearest' # نحوه پر کردن نواحی خالی
)

# flow() → (1, height, width, channels) تبدیل تصویر به شکل مناسب برای استفاده در 4
image = image.reshape((1,) + image.shape)

# generator تولید ۵ تصویر افزوده‌شده با استفاده از 5
augmented_images = []
for _ in range(5):
    augmented_batch = datagen.flow(image, batch_size=1)
    augmented_images.append(augmented_batch[0][0])

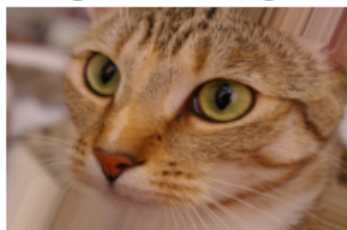
# نمایش تصاویر افزوده‌شده به‌صورت کنار هم 6
plt.figure(figsize=(15, 6))
for i, augmented_image in enumerate(augmented_images):
    plt.subplot(1, 5, i + 1)
    plt.imshow(augmented_image.astype(np.uint8))
    plt.title(f'Augmented Image {i+1}')
```

```
plt.axis('off')
plt.show()
```

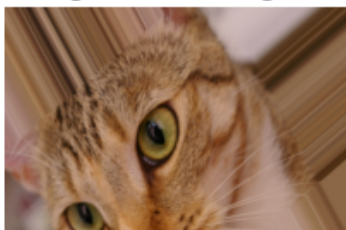
Original Image



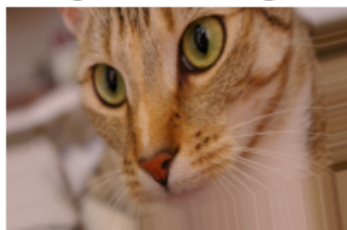
Augmented Image 1



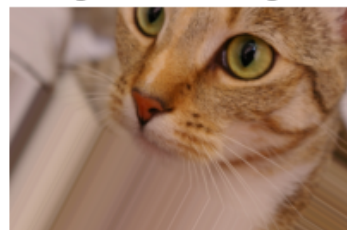
Augmented Image 2



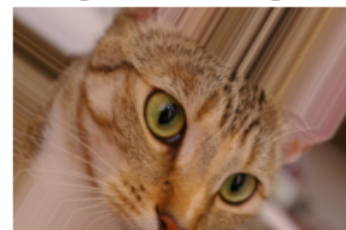
Augmented Image 3



Augmented Image 4



Augmented Image 5



افزایش داده‌های تصویری با ImageDataGenerator در Keras 🐱🧪

در این مثال، یک تصویر نمونه از گربه با استفاده از ابزار `ImageDataGenerator` در Keras به چندین نسخه‌ی افزوده‌شده تبدیل شده است. مراحل انجام‌شده به شرح زیر است:

- تصویر اولیه از کتابخانه `skimage` بارگذاری شده و به‌صورت RGB نمایش داده شده است.
- با استفاده از پارامترهای چرخش، جابجایی، برش، بزرگ‌نمایی و قرینه‌سازی، شیء `ImageDataGenerator` تعریف شده است.
- تصویر به شکل مناسب برای ورودی `generator` تبدیل شده و ۵ تصویر افزوده‌شده تولید شده‌اند.
- تصاویر نهایی به‌صورت کنار هم نمایش داده شده‌اند تا تأثیر تبدیلات قابل مشاهده باشد.

افزایش داده‌های تصویری یکی از تکنیک‌های کلیدی در یادگیری عمیق برای مقابله با کمبود داده، جلوگیری از `overfitting`، و بهبود تعمیم‌پذیری مدل‌هاست. این روش در مسائل بینایی ماشین مانند تشخیص چهره، طبقه‌بندی تصاویر، و تشخیص اشیاء بسیار کاربردی است.

12.3.3. From correlation to causality

حرکت از همبستگی به علیت (Moving from Correlation to Causality) در کاوش داده‌ها 🧠

حرکت از همبستگی (`Correlation`) به علیت (`Causality`) در کاوش داده‌ها شامل فراتر رفتن از شناسایی روابط بین متغیرها برای درک روابط علت و معلولی است که بر داده‌ها حاکم هستند. در حالی که همبستگی ارتباط آماری بین متغیرها را اندازه‌گیری می‌کند، اثبات علیت نیازمند تحلیل دقیق‌تر و اغلب شامل آزمایش یا روش‌های آماری پیشرفته است. تشخیص علیت برای اتخاذ تصمیمات آگاهانه و پیش‌بینی‌ها در حوزه‌های مختلف حیاتی است.

نکات کلیدی درباره حرکت از همبستگی به علیت:

- همبستگی در برابر علیت (Correlation vs. Causation):** همبستگی نشان‌دهنده یک ارتباط آماری بین دو متغیر است، اما به معنای علیت نیست. اثبات علیت شامل نشان دادن این است که تغییرات در یک متغیر به طور مستقیم بر تغییرات در متغیر دیگری تأثیر می‌گذارد.
- طراحی تجربی (Experimental Design):** آزمایش‌های کنترل‌شده اغلب برای اثبات علیت به کار می‌روند. آزمایش‌های تصادفی کنترل‌شده (Randomized Controlled Trials - RCTs) شامل دستکاری یک متغیر در حالی که بقیه ثابت نگه داشته می‌شوند، برای مشاهده اثر علی هستند.
- تحلیل پادواقعی (Counterfactual Analysis):** تحلیل پادواقعی، نتایج مشاهده‌شده را با آنچه در غیاب یک مداخله خاص اتفاق می‌افتاد مقایسه می‌کند. این رویکرد به تخصیص تغییرات به یک علت خاص کمک می‌کند.
- روش‌های استنباط علی (Causal Inference Methods):** روش‌های آماری پیشرفته، مانند تحلیل متغیر ابزاری (Instrumental Variable Analysis) و چارچوب‌های استنباط علی (Causal Inference Frameworks)، برای استنباط علیت از داده‌های مشاهده‌ای استفاده می‌شوند. این روش‌ها با هدف در نظر گرفتن عوامل مخدوش‌کننده (Confounding Factors) که ممکن است هم بر علت و هم بر معلول تأثیر بگذارند، عمل می‌کنند.
- کاربردهای حوزه‌ای (Domain Applications):** حرکت از همبستگی به علیت در زمینه‌هایی مانند مراقبت‌های بهداشتی، اقتصاد و علوم اجتماعی مرتبط است. درک روابط علی بین متغیرها برای طراحی مداخلات و سیاست‌های مؤثر ضروری است.

یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای کاوش علیت با استفاده از یک شبیه‌سازی ساده از یک آزمایش A/B (A/B Testing Simulation):

In [38]: # t-test و بررسی تأثیر مداخله با استفاده از A/B آموزش: شبیه‌سازی آزمون

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import ttest_ind

# 1 برای تولید عدد تصادفی قابل تکرار seed تنظیم
np.random.seed(42)

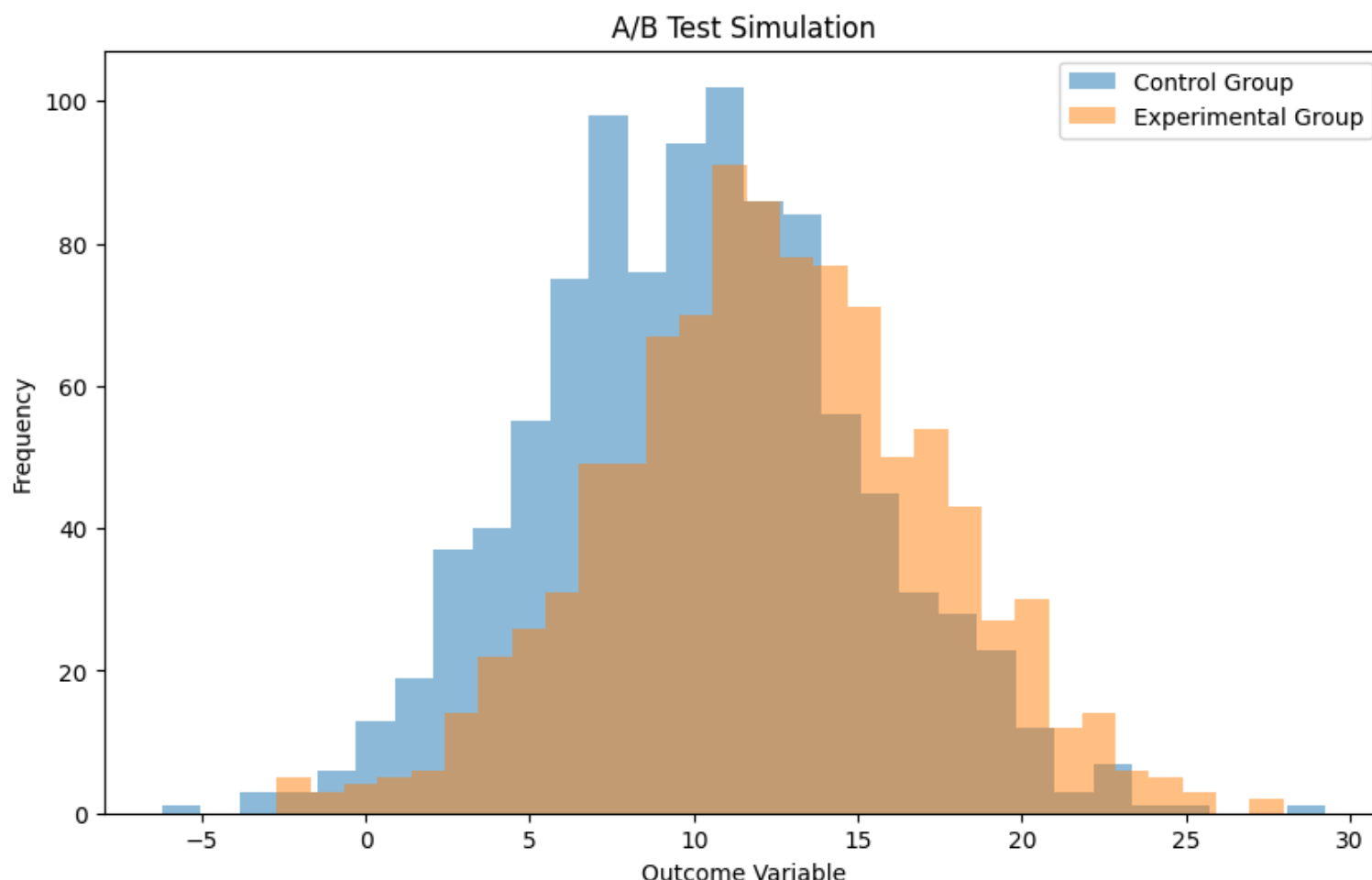
# 2 تولید داده‌های گروه کنترل (بدون مداخله)
control_group = np.random.normal(loc=10, scale=5, size=1000)

# 3 تولید داده‌های گروه آزمایشی (با مداخله)
experimental_group = np.random.normal(loc=12, scale=5, size=1000)

# 4 برای بررسی تفاوت میانگین‌ها t-test اجرای آزمون آماری
t_stat, p_value = ttest_ind(control_group, experimental_group)

# 5 رسم هیستوگرام برای نمایش توزیع دو گروه
plt.figure(figsize=(10, 6))
plt.hist(control_group, bins=30, alpha=0.5, label='Control Group')
plt.hist(experimental_group, bins=30, alpha=0.5, label='Experimental Group')
plt.title('A/B Test Simulation')
plt.xlabel('Outcome Variable')
plt.ylabel('Frequency')
plt.legend()
plt.show()

# 6 نمایش نتایج آزمون آماری
print(f'T-test results: t-statistic = {t_stat:.2f}, p-value = {p_value:.4f}')
```



T-test results: t-statistic = -10.21, p-value = 0.0000

شبیه‌سازی آزمون A/B و تحلیل آماری با t-test

در این مثال، یک شبیه‌سازی از آزمون A/B برای بررسی تأثیر مداخله روی متغیر خروجی انجام شده است. مراحل تحلیل به شرح زیر است:

- ابتدا دو گروه داده با توزیع نرمال تولید شده‌اند: گروه کنترل با میانگین ۱۰ و گروه آزمایشی با میانگین ۱۲.
- با استفاده از آزمون **t-test**، تفاوت میانگین دو گروه بررسی شده تا مشخص شود آیا مداخله تأثیر معناداری داشته است.
- نتایج آزمون شامل آماره t و مقدار p نمایش داده شده‌اند. مقدار p پایین‌تر از ۰.۰۵ نشان‌دهنده تفاوت معنادار آماری است.
- هیستوگرام دو گروه رسم شده تا توزیع داده‌ها به صورت بصری قابل مقایسه باشد.

این روش برای ارزیابی اثربخشی تغییرات در طراحی محصول، کمپین‌های بازاریابی، یا آزمایش‌های بالینی بسیار کاربردی است. می‌توان آن را با تحلیل‌های تکمیلی مانند اندازه اثر، آزمون‌های ناپارامتری، یا تحلیل واریانس ترکیب کرد تا نتایج دقیق‌تری حاصل شود.

12.3.4. Network as a context

استفاده از شبکه به عنوان یک زمینه (context) در کاوش داده‌ها

در نظر گرفتن شبکه به عنوان یک زمینه در کاوش داده‌ها شامل اذعان به ماهیت به‌هم‌پیوسته داده‌ها است، جایی که روابط و تعاملات بین موجودیت‌ها نقشی حیاتی ایفا می‌کنند. شبکه‌ها می‌توانند نمایانگر اتصالات اجتماعی، جریان اطلاعات، وابستگی‌ها یا هر شکلی از ساختار رابطه‌ای در داده‌ها باشند. تحلیل داده‌ها در بستر شبکه‌ها به درک عمیق‌تری از اینکه موجودیت‌ها چگونه بر اتصالات خود تأثیر می‌گذارند و از آن‌ها تأثیر می‌پذیرند، منجر می‌شود.

نکات کلیدی درباره استفاده از شبکه به عنوان یک زمینه:

- **نمایش شبکه (Network Representation):** داده‌ها می‌توانند به عنوان یک شبکه نمایش داده شوند، جایی که موجودیت‌ها (گره‌ها - nodes) توسط روابط یا تعاملات (یال‌ها - edges) به هم متصل می‌شوند. این نمایش برای حوزه‌های مختلفی از جمله شبکه‌های اجتماعی، سیستم‌های بیولوژیکی و جریان اطلاعات قابل استفاده است.
- **بینش‌های زمینه‌ای (Contextual Insights):** تحلیل داده‌ها در بستر یک شبکه، بینش‌هایی درباره نفوذ و تأثیر روابط فراهم می‌کند. این امر به درک جامع‌تری از اینکه موجودیت‌ها چگونه در محیط به‌هم‌پیوسته خود عمل می‌کنند، کمک می‌کند.
- **تشخیص جامعه (Community Detection):** الگوریتم‌های تشخیص جامعه در شبکه‌ها، گروه‌هایی از موجودیت‌ها را که به شدت در داخل خود به هم متصل هستند، آشکار می‌کنند. این به شناسایی زیرساختارها و واحدهای عملکردی در داخل داده‌ها کمک می‌کند.
- **تحلیل مرکزیت (Centrality Analysis):** معیارهای مرکزیت، مانند مرکزیت درجه (degree centrality) یا مرکزیت بینابینی (betweenness centrality)، اهمیت گره‌ها در شبکه را برجسته می‌کنند. این تحلیل، موجودیت‌های کلیدی را که نقش‌های مهمی در جریان اطلاعات یا نفوذ ایفا می‌کنند، شناسایی می‌نماید.
- **جنبه‌های زمانی (Temporal Aspects):** در نظر گرفتن جنبه‌های زمانی داده‌های شبکه یک لایه اضافی از پیچیدگی را اضافه می‌کند. تحلیل اینکه روابط چگونه در طول زمان تکامل می‌یابند، یک دیدگاه پویا درباره داده‌ها فراهم می‌کند.

یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای تحلیل یک شبکه اجتماعی با استفاده از تکنیک‌های تحلیل شبکه:

In [45]: *و مرکزیت درجه Louvain آموزش: تحلیل شبکه اجتماعی با تشخیص اجتماع #*

```
import networkx as nx
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import community.community_louvain as community_louvain # ایمپورت صحیح کتابخانه
```

```
# ساخت شبکه نمونه با گره‌های اجتماعی و روابط بین آن‌ها 1
G = nx.Graph()
edges = [
    ('Alice', 'Bob'), ('Bob', 'Charlie'), ('Charlie', 'Alice'),
    ('Alice', 'David'), ('David', 'Eve'), ('Eve', 'Frank'),
    ('Frank', 'Charlie'), ('Bob', 'David')
]
G.add_edges_from(edges)
```

```
# محاسبه مرکزیت درجه برای سنجش میزان مشارکت هر گره 2
degree_centrality = nx.degree_centrality(G)
```

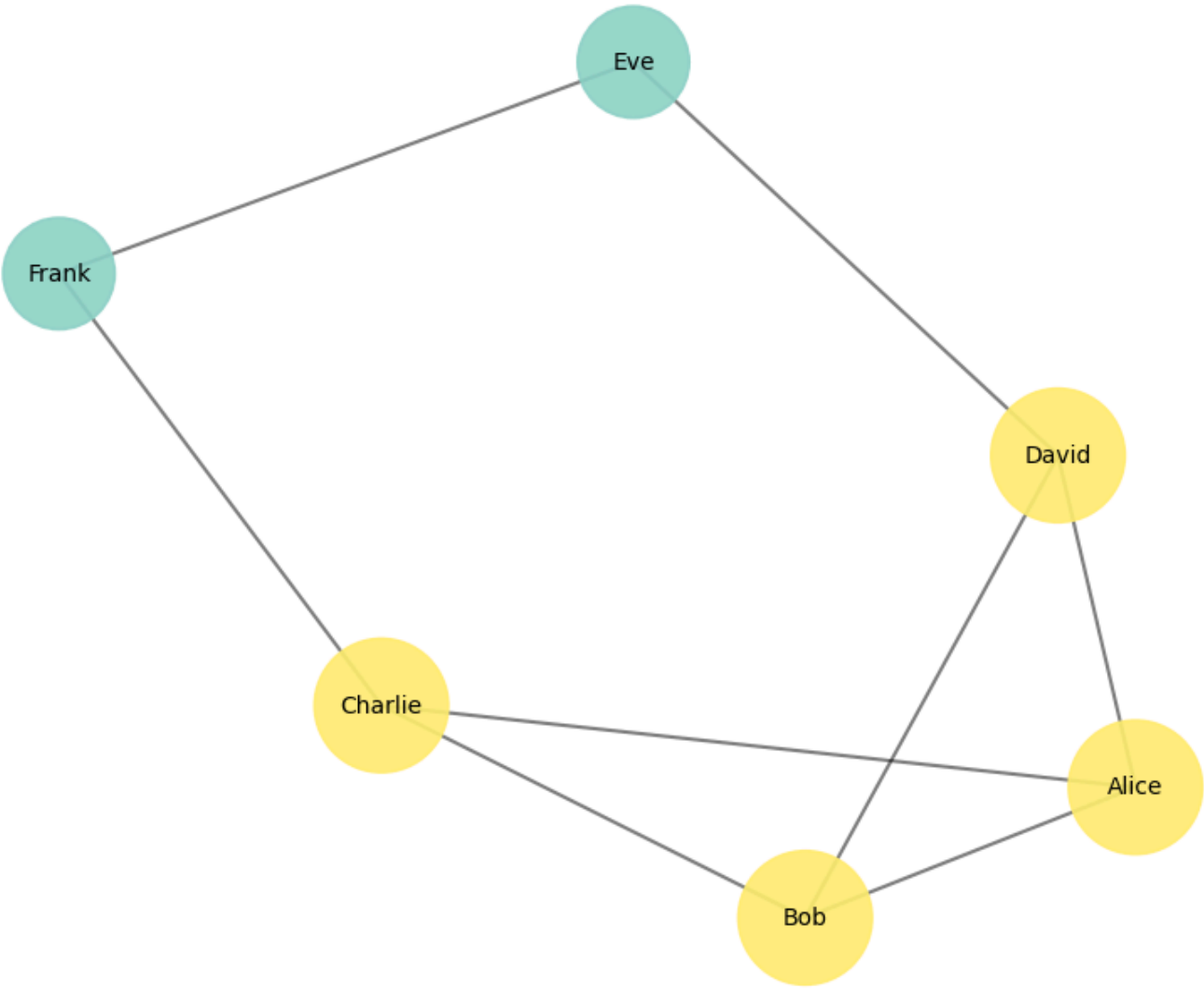
```
# Louvain تشخیص اجتماع‌ها با استفاده از الگوریتم 3
partition = community_louvain.best_partition(G)
```

```
# آماده‌سازی رنگ گره‌ها بر اساس اجتماع و اندازه گره‌ها بر اساس مرکزیت 4
cmap = cm.get_cmap('Set3', max(partition.values()) + 1)
node_colors = [cmap(partition[node]) for node in G.nodes()]
node_sizes = [5000 * degree_centrality[node] + 300 for node in G.nodes()]
```

```
# رسم گراف با رنگ‌بندی اجتماع‌ها و اندازه متناسب با مرکزیت 5
plt.figure(figsize=(10, 8))
pos = nx.spring_layout(G, seed=42)
nx.draw_networkx_nodes(G, pos, node_size=node_sizes, node_color=node_colors, alpha=0.9)
nx.draw_networkx_edges(G, pos, width=1.5, alpha=0.5)
nx.draw_networkx_labels(G, pos, font_size=10, font_color='black')
plt.title("Social Network with Centrality and Communities", fontsize=16)
plt.axis('off')
plt.show()
```

```
# نمایش جدول اطلاعات گره‌ها شامل مرکزیت و شماره اجتماع 6
print("Node Centrality and Community:")
for node in G.nodes():
    print(f"{node}: Degree={degree_centrality[node]:.2f}, Community={partition[node]}")
```

/tmp/ipython-input-2348205957.py:24: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use `matplotlib.colormaps[name]` or `matplotlib.colormaps.get\_cmap()` or `pyplot.get\_cmap()` instead.
cmap = cm.get_cmap('Set3', max(partition.values()) + 1)



Node Centrality and Community:
Alice: Degree=0.60, Community=1
Bob: Degree=0.60, Community=1
Charlie: Degree=0.60, Community=1
David: Degree=0.60, Community=1
Eve: Degree=0.40, Community=0
Frank: Degree=0.40, Community=0

📡👥 تحلیل شبکه اجتماعی با تشخیص اجتماع Louvain و مرکزیت درجه

در این پروژه، یک شبکه اجتماعی ساده مدل‌سازی شده و با استفاده از الگوریتم Louvain، اجتماع‌های درون شبکه شناسایی شده‌اند. همچنین، مرکزیت درجه برای سنجش میزان مشارکت هر گره محاسبه شده است. مراحل انجام‌شده به شرح زیر است:

- شبکه شامل گره‌هایی مانند Alice، Bob و Charlie است که با روابط همکاری یا ارتباط به هم متصل شده‌اند.
- مرکزیت درجه برای هر گره محاسبه شده تا میزان مشارکت آن در شبکه مشخص شود.
- الگوریتم Louvain برای تشخیص اجتماع‌ها اجرا شده و هر گره به یک گروه تعلق گرفته است.
- رنگ گره‌ها بر اساس اجتماع و اندازه آن‌ها بر اساس مرکزیت درجه تنظیم شده‌اند.
- نقشه شبکه رسم شده و جدول نهایی شامل اطلاعات عددی هر گره نمایش داده شده است.

این تحلیل برای بررسی ساختار گروهی در شبکه‌های اجتماعی، شناسایی افراد کلیدی، و تحلیل رفتار جمعی بسیار کاربردی است. می‌توان آن را با داده‌های واقعی از شبکه‌های ارتباطی، سازمانی یا علمی ترکیب کرد تا تحلیل‌های عمیق‌تری ارائه شود.

AutoML، یا Automated Machine Learning، به توسعه روش‌ها و سیستم‌هایی اشاره دارد که فرآیند سرتاسری (End-to-End) اعمال یادگیری ماشین (Machine Learning) بر روی مسائل دنیای واقعی را خودکار می‌کنند. AutoML با هدف دسترس‌پذیر کردن یادگیری ماشین برای افراد غیرمتخصص و ساده‌سازی فرآیند توسعه مدل از طریق خودکارسازی وظایفی مانند مهندسی ویژگی (Feature Engineering)، انتخاب الگوریتم (Algorithm Selection) و تنظیم فرآپارامتر (Hyperparameter Tuning) است. این روند در روش‌شناسی‌های کاوش داده‌ها به دلیل پتانسیل خود در دموکراتیزه کردن یادگیری ماشین و تسریع استقرار مدل اهمیت یافته است.

نکات کلیدی درباره روش‌ها و سیستم‌های AutoML:

- انتخاب خودکار مدل (Automated Model Selection):** سیستم‌های AutoML به طور خودکار مناسب‌ترین الگوریتم‌های یادگیری ماشین را برای یک وظیفه مشخص، با در نظر گرفتن ویژگی‌های داده، کاوش و انتخاب می‌کنند.
- بهینه‌سازی فرآپارامتر (Hyperparameter Optimization):** بهینه‌سازی فرآپارامترها، که به طور قابل توجهی بر عملکرد مدل تأثیر می‌گذارند، برای یافتن بهترین پیکربندی (Configuration) برای یک الگوریتم مشخص به صورت خودکار انجام می‌شود.
- خودکارسازی مهندسی ویژگی (Feature Engineering Automation):** مهندسی ویژگی، فرآیند ایجاد ویژگی‌های مرتبط برای آموزش مدل، به صورت خودکار انجام می‌شود تا تلاش دستی مورد نیاز در پیش‌پردازش داده‌ها کاهش یابد.
- ساخت خط لوله (Pipeline Construction):** ابزارهای AutoML خطوط لوله (Pipelines) سرتاسری یادگیری ماشین را می‌سازند که شامل پیش‌پردازش داده‌ها، مهندسی ویژگی، آموزش مدل و ارزیابی است.
- مقیاس‌پذیری و کارایی (Scalability and Efficiency):** سیستم‌های AutoML برای مدیریت کارآمد مجموعه‌های داده بزرگ طراحی شده‌اند و می‌توانند برای پاسخگویی به تقاضاهای وظایف پیچیده یادگیری ماشین مقیاس‌پذیر باشند.
- دموکراتیزه کردن یادگیری ماشین (Democratizing Machine Learning):** با خودکارسازی جنبه‌های پیچیده یادگیری ماشین، AutoML دسترسی به قابلیت‌های یادگیری ماشین را دموکراتیزه می‌کند و افراد با سطوح مختلف تخصص را قادر می‌سازد تا از تحلیل‌های پیشرفته بهره ببرند.

یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای AutoML با استفاده از کتابخانه **tpot (simulated!)**:

In []: `!pip install tpot`

In [10]: *روی داده‌های ارقام TPOT با AutoML آموزش: پیاده‌سازی شبیه‌سازی‌شده از خروجی #*

```
import numpy as np
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.datasets import load_digits
from sklearn.metrics import accuracy_score

# 1 بارگذاری داده‌های ارقام از sklearn
digits = load_digits()
X_train, X_test, y_train, y_test = train_test_split(
    digits.data, digits.target, random_state=42
)

# شامل TPOT ساخت پایپ‌لاین شبیه‌سازی‌شده از خروجی 2
# با پارامترهای تنظیم‌شده Gradient Boosting استانداردسازی داده‌ها + مدل
pipeline = make_pipeline(
    StandardScaler(),
    GradientBoostingClassifier(
        learning_rate=0.1,
        max_depth=5,
        max_features=0.8,
        min_samples_leaf=3,
        min_samples_split=5,
        n_estimators=100,
        subsample=0.9,
        random_state=42
    )
)

# آموزش مدل روی داده‌های آموزشی 3
pipeline.fit(X_train, y_train)

# پیش‌بینی روی داده‌های تست 4
```

```
predictions = pipeline.predict(X_test)

# ارزیابی دقت مدل با معیار accuracy
accuracy = accuracy_score(y_test, predictions)
print(f"AutoML (simulated TPOT pipeline) Accuracy: {accuracy:.4f}")
```

AutoML (simulated TPOT pipeline) Accuracy: 0.9756

پیاده‌سازی شبیه‌سازی‌شده از خروجی AutoML با TPOT روی داده‌های ارقام



در این پروژه، یک پایپ‌لاین یادگیری ماشین به‌صورت شبیه‌سازی‌شده از خروجی AutoML توسط ابزار TPOT طراحی شده و روی مجموعه داده‌های ارقام (digits) اجرا شده است. مراحل انجام‌شده به شرح زیر است:

- داده‌های ارقام از مجموعه `sklearn.datasets` بارگذاری شده و به دو بخش آموزش و تست تقسیم شده‌اند.
- پایپ‌لاین شامل دو مرحله است: استانداردسازی ویژگی‌ها با `StandardScaler` و آموزش مدل `GradientBoostingClassifier` با پارامترهای بهینه‌شده.
- مدل روی داده‌های آموزشی آموزش داده شده و سپس روی داده‌های تست پیش‌بینی انجام شده است.
- دقت مدل با استفاده از معیار `accuracy_score` محاسبه شده و به‌صورت عددی نمایش داده شده است.

این نوع پایپ‌لاین‌ها در AutoML برای یافتن بهترین ترکیب پیش‌پردازش و مدل‌سازی به‌صورت خودکار استفاده می‌شوند. ابزارهایی مانند H2O، TPOT، و Auto-sklearn می‌توانند با جستجوی هوشمندانه، مدل‌هایی با عملکرد بالا تولید کنند. این رویکرد برای مسائل طبقه‌بندی، پیش‌بینی عددی، و تحلیل داده‌های پیچیده بسیار کاربردی است.

Section 12.4

12.4.1. Privacy-preserving data mining

🔒 کاوش داده‌ها با حفظ حریم خصوصی (Privacy-preserving data mining)

کاوش داده‌ها با حفظ حریم خصوصی به چالش استخراج بینش‌های ارزشمند از داده‌ها در عین محافظت از حریم خصوصی افرادی می‌پردازد که اطلاعاتشان بخشی از مجموعه داده است. با پیشرفته‌تر شدن تکنیک‌های کاوش داده‌ها، نیاز فزاینده‌ای به توسعه روش‌هایی وجود دارد که از افشای اطلاعات حساس جلوگیری کنند. تکنیک‌های کاوش داده‌ها با حفظ حریم خصوصی، تحلیل داده‌ها را بدون به خطر انداختن محرمانگی جزئیات شخصی امکان‌پذیر می‌سازند و استفاده اخلاقی و مسئولانه از داده‌ها را ترویج می‌دهند.

نکات کلیدی درباره کاوش داده‌ها با حفظ حریم خصوصی:

- حفاظت از داده‌های حساس (Sensitive Data Protection):** کاوش داده‌ها با حفظ حریم خصوصی با هدف حفاظت از اطلاعات حساس در داخل مجموعه‌های داده، مانند شناسه‌های شخصی، سوابق بهداشتی یا جزئیات مالی، صورت می‌گیرد.
- تکنیک‌های رمزنگاری (Cryptographic Techniques):** روش‌های رمزنگاری، مانند رمزنگاری هم‌ریخت (homomorphic encryption) و محاسبات امن چندطرفه (secure multi-party computation)، برای انجام محاسبات بر روی داده‌های رمزنگاری‌شده بدون افشای اطلاعات زیربنایی به کار می‌روند.

- **حریم خصوصی دیفرانسیلی (Differential Privacy):** حریم خصوصی دیفرانسیلی تضمین می‌کند که افزودن یا حذف داده‌های یک فرد، به طور قابل توجهی بر نتیجه الگوریتم‌های کاوش داده‌ها تأثیر نمی‌گذارد. این امر نویز یا تصادفی بودن را به داده‌ها اضافه می‌کند تا از شناسایی مجدد (re-identification) جلوگیری شود.
- **گمنام‌سازی و بی‌هویت‌سازی (Anonymization and De-identification):** تکنیک‌هایی مانند گمنام‌سازی (anonymization) و بی‌هویت‌سازی (de-identification) شامل حذف یا اصلاح اطلاعات شخصی برای کاهش خطر شناسایی افراد هستند.

- **ملاحظات اخلاقی (Ethical Considerations):** کاوش داده‌ها با حفظ حریم خصوصی توسط ملاحظات اخلاقی هدایت می‌شود، با هدف ایجاد تعادل بین مزایای تحلیل داده‌ها و حفاظت از حقوق حریم خصوصی افراد.

یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای کاوش داده‌ها با حفظ حریم خصوصی با استفاده از کتابخانه **diffprivlib** برای حریم خصوصی دیفرانسیلی:

```
In [ ]: !pip install diffprivlib
```

```
In [16]: # روی داده‌های DiffprivLib آموزش: مقایسه رگرسیون لجستیک معمولی و حفظ حریم خصوصی با Iris
```

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from diffprivlib.models import LogisticRegression as DiffPrivLogisticRegression
from sklearn.metrics import accuracy_score
```

```
# 1 بارگذاری داده‌های کلاسیک Iris و تقسیم به آموزش و تست
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)
```

```
# 2 آموزش مدل رگرسیون لجستیک معمولی بدون حفظ حریم خصوصی
normal_model = LogisticRegression()
normal_model.fit(X_train, y_train)
```

```
# 3 پیش‌بینی روی داده‌های تست با مدل معمولی
normal_predictions = normal_model.predict(X_test)
```

```
# 4 محاسبه دقت مدل معمولی
normal_accuracy = accuracy_score(y_test, normal_predictions)
print(f"Normal Model Accuracy: {normal_accuracy:.4f}")
```

```
# 5 آموزش مدل رگرسیون لجستیک با حفظ تفاضلی حریم خصوصی (ε=1.0)
privacy_model = DiffPrivLogisticRegression(
    epsilon=1.0,
    data_norm=np.linalg.norm(X_train) # نرم داده‌ها برای تنظیم نویز
)
privacy_model.fit(X_train, y_train)
```

```
# 6 پیش‌بینی روی داده‌های تست با مدل حفظ حریم خصوصی
privacy_predictions = privacy_model.predict(X_test)
```

```
# 7 محاسبه دقت مدل حفظ حریم خصوصی
privacy_accuracy = accuracy_score(y_test, privacy_predictions)
print(f"Privacy-Preserving Model Accuracy: {privacy_accuracy:.4f}")
```

Normal Model Accuracy: 1.0000

Privacy-Preserving Model Accuracy: 0.3000

مقایسه رگرسیون لجستیک معمولی و حفظ حریم خصوصی با Diffprivlib



در این پروژه، دو مدل رگرسیون لجستیک روی داده‌های Iris آموزش داده شده‌اند: یکی بدون حفظ حریم خصوصی و دیگری با استفاده از الگوریتم‌های **حفظ تفاضلی حریم خصوصی** از کتابخانه **diffprivlib**. مراحل انجام‌شده به شرح زیر است:

- داده‌های Iris بارگذاری شده و به دو بخش آموزش و تست تقسیم شده‌اند.
- مدل اول با استفاده از **LogisticRegression** معمولی آموزش داده شده و دقت آن محاسبه شده است.

- این مقایسه نشان می‌دهد که حفظ حریم خصوصی با مکانیزم تفاضلی ممکن است منجر به کاهش جزئی در دقت شود، اما در کاربردهایی مانند تحلیل داده‌های حساس (پزشکی، مالی، آموزشی) بسیار ضروری و ارزشمند است. می‌توان با تنظیم دقیق پارامتر `epsilon` و نُرم داده‌ها، تعادل مناسبی بین دقت و حفظ حریم خصوصی برقرار کرد.

- استفاده از `epsilon=1.0` مقدار زیادی نویز اضافه می‌کند که منجر به افت دقت می‌شود.
- مجموعه داده `Iris` کوچک است (۱۵۰ نمونه) و نویز روی داده‌های کم، اثر قابل توجهی دارد
- مقدار نُرم داده‌ها (`data_norm`) اگر بزرگ یا اشتباه انتخاب شود، نویز بیشتری اعمال می‌کند

برای بهبود دقت مدل حفظ حریم خصوصی می‌توان اقدامات زیر را انجام داد:

- افزایش `epsilon` (مثلاً ۵ یا ۱۰) تا نویز کاهش یابد و دقت افزایش یابد.
- استفاده از `data_norm=np.max(np.linalg.norm(X_train, axis=1))` برای کاهش اثر نویز روی نمونه‌ها.
- افزایش تعداد نمونه‌ها یا استفاده از مجموعه داده بزرگ‌تر.
- تنظیم تعداد تکرارهای آموزش با `max_iter` مناسب برای همگرایی بهتر مدل.

با این اقدامات می‌توان تعادل مناسبی بین **حفظ حریم خصوصی** و **دقت مدل** برقرار کرد، به‌طوری که همچنان از داده‌های حساس محافظت شود و عملکرد مدل نیز قابل قبول باشد.

🧡 تعامل انسان و الگوریتم (Human-Algorithm Interaction)

تعامل انسان و الگوریتم به بررسی کنش و واکنش پویا بین انسان و الگوریتم‌ها در بستر کاوش داده‌ها و یادگیری ماشین (Machine Learning) می‌پردازد. این روند، نقش محوری مشارکت انسان در فرآیند کاوش داده‌ها را تأیید کرده و بر همکاری، قابلیت تفسیر (Interpretability) و واسطه‌های کاربرپسند (User-Friendly Interfaces) تأکید می‌کند. با تقویت یک رابطه

همزیستی بین انسان و الگوریتم‌ها، این رویکرد با هدف افزایش اثربخشی، شفافیت و ملاحظات اخلاقی در تصمیم‌گیری مبتنی بر داده‌ها صورت می‌گیرد.

نکات کلیدی درباره تعامل انسان و الگوریتم:

- تصمیم‌گیری مشارکتی (Collaborative Decision-Making):** تعامل انسان و الگوریتم، همکاری بین دانشمندان داده، متخصصان حوزه (Domain Experts) و کاربران نهایی را برای اتخاذ جمعی تصمیمات و استخراج بینش‌های معنادار از داده‌ها ترویج می‌دهد.
- قابلیت تفسیر و توضیح‌پذیری (Interpretability and Explainability):** الگوریتم‌ها به گونه‌ای طراحی می‌شوند که قابل تفسیر و توضیح‌پذیر باشند و کاربران را قادر می‌سازند تا تصمیمات گرفته شده توسط مدل‌ها را درک کرده و به آن‌ها اعتماد کنند. این امر به ویژه در حوزه‌های حیاتی مانند مراقبت‌های بهداشتی و مالی مهم است.
- واسط‌های کاربرپسند (User-Friendly Interfaces):** توسعه واسط‌های کاربرپسند، تعامل یکپارچه بین انسان و الگوریتم‌ها را تسهیل می‌کند و به کاربران با سطوح مختلف تخصص فنی اجازه می‌دهد تا به طور فعال در فرآیند تحلیل داده‌ها شرکت کنند.
- مکانیزم‌های بازخورد (Feedback Mechanisms):** گنجاندن مکانیزم‌های بازخورد، بهبود مستمر الگوریتم‌ها را بر اساس ورودی، ترجیحات و دانش خاص حوزه کاربران ممکن می‌سازد.
- ملاحظات اخلاقی (Ethical Considerations):** تعامل انسان و الگوریتم، ملاحظات اخلاقی را در نظر می‌گیرد و اطمینان می‌دهد که فرآیندهای تصمیم‌گیری عادلانه، شفاف و همسو با ارزش‌های اجتماعی هستند.

یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای تعامل انسان و الگوریتم با استفاده از یک کتابخانه بصری‌سازی تعاملی:

In [17]: *# Plotly و نمایش تعاملی با PCA آموزش: کاهش ابعاد داده‌های گل زنبق با*

```
import plotly.express as px
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA

# 1 بارگذاری داده‌های کلاسیک گل زنبق (Iris)
iris = load_iris()
X = iris.data
y = iris.target

# 2 به دو مؤلفه اصلی PCA اعمال کاهش ابعاد با الگوریتم
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# 3 و نام گونه‌ها PCA ساخت دیتافریم شامل مؤلفه‌های
df = pd.DataFrame({
    'PCA1': X_pca[:, 0],
    'PCA2': X_pca[:, 1],
    'Species': iris.target_names[y]
})

# 4 Plotly Express رسم نمودار تعاملی پراکندگی با
fig = px.scatter(
    df,
    x='PCA1',
    y='PCA2',
    color='Species',
    title='Interactive PCA Plot'
)
fig.show()
```

کاهش ابعاد داده‌های گل زنبق با PCA و نمایش تعاملی

در این پروژه، داده‌های کلاسیک گل زنبق (Iris) با استفاده از الگوریتم PCA به دو مؤلفه اصلی کاهش یافته‌اند و با ابزار Plotly به صورت تعاملی نمایش داده شده‌اند. مراحل انجام‌شده به شرح زیر است:

- داده‌های شامل ویژگی‌های طول و عرض گلبرگ و کاسبرگ برای سه گونه مختلف بارگذاری شده‌اند.
- با استفاده از الگوریتم **Principal Component Analysis**، داده‌ها به دو مؤلفه اصلی کاهش یافته‌اند تا ساختار پنهان قابل مشاهده شود.
- نمودار پراکندگی با رنگ‌بندی بر اساس گونه‌ها رسم شده تا جدایی کلاس‌ها به صورت بصری قابل تحلیل باشد.
- استفاده از Plotly Express امکان تعامل با نمودار، بزرگ‌نمایی، و مشاهده جزئیات را فراهم کرده است.

این روش برای تحلیل اکتشافی داده‌ها، درک ساختار درونی کلاس‌ها، و آماده‌سازی داده‌ها برای مدل‌های طبقه‌بندی بسیار کاربردی است. می‌توان آن را با الگوریتم‌های کاهش ابعاد دیگر مانند t-SNE یا UMAP نیز مقایسه کرد.

12.4.3. Mining beyond maximizing accuracy: fairness, interpretability, and robustness

کاوش فراتر از حداکثرسازی دقت (Mining beyond maximizing accuracy)

کاوش داده‌ها فراتر از حداکثرسازی دقت (accuracy) شامل در نظر گرفتن عوامل اضافی مانند انصاف (fairness)، قابلیت تفسیر (interpretability) و استحکام (robustness) است. در حالی که دقت ضروری است، ممکن است تنها معیار برای ارزیابی عملکرد یک مدل نباشد، به ویژه در سناریوهایی که ملاحظات اخلاقی، درک انسانی و مقاومت در برابر حملات خصمانه (adversarial attacks) حیاتی هستند. این روند بر اهمیت توسعه مدل‌هایی که در کاربردهای دنیای واقعی منصفانه، قابل تفسیر و مستحکم هستند، تأکید می‌کند.

نکات کلیدی درباره کاوش فراتر از حداکثرسازی دقت:

- **انصاف (Fairness):** انصاف در کاوش داده‌ها شامل اطمینان از این است که مدل‌ها سوگیری‌هایی (biases) از خود نشان نمی‌دهند که می‌تواند به نتایج تبعیض‌آمیز منجر شود، به ویژه برای گروه‌های جمعیتی مختلف. معیارهای انصاف و الگوریتم‌ها برای کاهش و اندازه‌گیری سوگیری استفاده می‌شوند.
- **قابلیت تفسیر (Interpretability):** مدل‌های قابل تفسیر به گونه‌ای طراحی شده‌اند که توضیحات واضحی از تصمیمات خود ارائه دهند. این امر در کاربردهایی که شفافیت مورد نیاز است، مانند مراقبت‌های بهداشتی یا مالی، برای ایجاد اعتماد و تسهیل تصمیم‌گیری حیاتی است.
- **استحکام (Robustness):** مدل‌های مستحکم در برابر حملات خصمانه و تغییرات در داده ورودی مقاوم هستند. اطمینان از استحکام در کاربردهایی که عملکرد مدل می‌تواند تحت تأثیر دستکاری عمدی یا تغییرات پیش‌بینی‌نشده در توزیع داده‌ها قرار گیرد، مهم است.
- **ملاحظات اخلاقی (Ethical Considerations):** کاوش فراتر از دقت بر ملاحظات اخلاقی در توسعه و استقرار مدل‌های کاوش داده‌ها تأکید می‌کند. این شامل پرداختن به سوگیری‌ها، تضمین شفافیت و اولویت‌بندی انصاف در تصمیم‌گیری است.

یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای تضمین انصاف در یک مدل یادگیری ماشین با استفاده از کتابخانه Fairness Indicators:

In []: !pip install fairlearn aif360

In []: !pip install 'aif360[inFairness]'

In [25]: *Adult Census روی داده‌های AIF360 آموزش: ارزیابی دقت و عدالت مدل طبقه‌بندی با استفاده از*

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# برای ارزیابی عدالت الگوریتمی AIF360 کتابخانه
from aif360.datasets import BinaryLabelDataset
from aif360.metrics import ClassificationMetric

# UCI بارگذاری داده‌های درآمد سرشماری بزرگسالان از 1
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data'
columns = ['age', 'workclass', 'fnlwgt', 'education', 'marital-status', 'occupation',
            'relationship', 'race', 'sex', 'capital-gain', 'capital-loss', 'hours-per-week',
            'native-country', 'income']
data = pd.read_csv(url, names=columns, delimiter=r'\s*', na_values=['?'], engine='python')

# پیش‌پردازش داده‌ها: حذف مقادیر گمشده و تبدیل ویژگی‌ها به عددی 2
data = data.dropna()
X = pd.get_dummies(data.drop('income', axis=1))
y = (data['income'] == '>50K').astype(int) # 1 و 0 به برچسب تبدیل

# تقسیم داده‌ها به آموزش و تست 3
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# آموزش مدل جنگل تصادفی 4
model = RandomForestClassifier()
model.fit(X_train, y_train)
preds = model.predict(X_test)

# ارزیابی دقت مدل 5
accuracy = accuracy_score(y_test, preds)
print(f"Model Accuracy: {accuracy:.4f}")

# -----
# AIF360 ارزیابی عدالت الگوریتمی با 6
# -----

# one-hot encoding بعد از) تعریف ویژگی محافظت‌شده: جنسیت زن
protected_attr = 'sex_Female'

# برای داده‌های تست BinaryLabelDataset ساخت شیء
test_bld = BinaryLabelDataset(
    favorable_label=1,
    unfavorable_label=0,
    df=X_test.assign(income=y_test),
    label_names=['income'],
    protected_attribute_names=[protected_attr]
)

# افزودن پیش‌بینی‌ها به نسخه کپی‌شده از داده‌ها
test_bld_pred = test_bld.copy()
test_bld_pred.labels = preds.reshape(-1, 1)

# محاسبه معیارهای عدالت الگوریتمی
```

```
metric = ClassificationMetric(  
    test_bld, test_bld_pred,  
    unprivileged_groups=[{protected_attr: 0}],  
    privileged_groups=[{protected_attr: 1}]  
)  
  
print(f"Disparate Impact: {metric.disparate_impact():.4f}")  
print(f"Statistical Parity Difference: {metric.statistical_parity_difference():.4f}")  
print(f"Equal Opportunity Difference: {metric.equal_opportunity_difference():.4f}")
```

Model Accuracy: 0.8367
Disparate Impact: 3.0933
Statistical Parity Difference: 0.2018
Equal Opportunity Difference: 0.0808

ارزیابی دقت و عدالت الگوریتمی روی داده‌های درآمد سرشماری بزرگسالان



در این پروژه، یک مدل طبقه‌بندی با استفاده از جنگل تصادفی روی داده‌های Adult Census Income آموزش داده شده و علاوه بر دقت، عدالت الگوریتمی آن با استفاده از کتابخانه **AIF360** بررسی شده است. مراحل انجام‌شده به شرح زیر است:

- داده‌ها از منبع UCI بارگذاری شده و پس از حذف مقادیر گمشده، ویژگی‌ها به صورت عددی تبدیل شده‌اند.
- مدل RandomForestClassifier آموزش داده شده و دقت آن با معیار accuracy_score محاسبه شده است.
- ویژگی محافظت‌شده sex_Female برای ارزیابی عدالت انتخاب شده است.
- با استفاده از BinaryLabelDataset و ClassificationMetric، سه معیار کلیدی عدالت محاسبه شده‌اند:
 - **Disparate Impact**: نسبت نرخ پیش‌بینی مطلوب بین گروه‌های محافظت‌شده و غیرمحافظت‌شده
 - **Statistical Parity Difference**: تفاوت نرخ پیش‌بینی مطلوب بین دو گروه
 - **Equal Opportunity Difference**: تفاوت نرخ پیش‌بینی درست برای موارد مطلوب بین دو گروه

این تحلیل برای شناسایی تبعیض الگوریتمی در مدل‌های یادگیری ماشین، به‌ویژه در حوزه‌هایی مانند استخدام، اعتبار مالی، یا خدمات عمومی بسیار حیاتی است. می‌توان آن را با روش‌های اصلاح مدل یا داده برای بهبود عدالت ترکیب کرد.

12.4.4. Data mining for social good

کاوش داده‌ها برای منفعت اجتماعی (Data Mining for Social Good) ❤️

کاوش داده‌ها برای منفعت اجتماعی شامل بهره‌گیری از تکنیک‌های کاوش داده‌ها برای پرداختن به چالش‌های اجتماعی و کمک به تأثیرگذاری مثبت اجتماعی است. با اعمال کاوش داده‌ها بر حوزه‌هایی مانند سلامت عمومی، آموزش، حفاظت از محیط زیست و تلاش‌های بشردوستانه، محققان و متخصصان می‌توانند بینش‌هایی را استخراج کنند که منجر به تصمیم‌گیری آگاهانه و راه‌حل‌های نوآورانه برای بهبود جامعه می‌شود.

نکات کلیدی درباره کاوش داده‌ها برای منفعت اجتماعی:

- **چالش‌های اجتماعی (Societal Challenges)**: کاوش داده‌ها برای منفعت اجتماعی بر مقابله با چالش‌های اجتماعی حیاتی مانند فقر، نابرابری‌های مراقبت‌های بهداشتی، پایداری محیطی و دسترسی به آموزش تمرکز دارد.

- **تصمیم‌گیری آگاهانه (Informed Decision-Making):** به کارگیری کاوش داده‌ها، تصمیم‌گیرندگان، سیاست‌گذاران و سازمان‌ها را قادر می‌سازد تا تصمیمات آگاهانه و مبتنی بر شواهد بگیرند و به مداخلات و استراتژی‌های مؤثرتری منجر شود.
- **تلاش‌های بشردوستانه (Humanitarian Efforts):** کاوش داده‌ها در تلاش‌های بشردوستانه برای بهینه‌سازی تخصیص منابع، برنامه‌ریزی پاسخ و مدیریت بلایا استفاده می‌شود. این امر نقشی حیاتی در بهبود کارایی و اثربخشی ارائه کمک‌ها ایفا می‌کند.
- **ابتکارات سلامت عمومی (Public Health Initiatives):** کاوش داده‌ها با تحلیل داده‌های مراقبت‌های بهداشتی برای شناسایی الگوهای بیماری، پیش‌بینی شیوع و بهینه‌سازی سیستم‌های ارائه مراقبت‌های بهداشتی به ابتکارات سلامت عمومی کمک می‌کند.
- **آموزش و برابری (Education and Equality):** در حوزه آموزش، کاوش داده‌ها می‌تواند برای رسیدگی به مسائل مربوط به دسترسی، کیفیت و برابری به کار گرفته شود، که به تنظیم مداخلات آموزشی بر اساس نیازهای فردی کمک می‌کند.
- **ملاحظات اخلاقی (Ethical Considerations):** کاوش داده‌ها برای منفعت اجتماعی بر ملاحظات اخلاقی تأکید می‌کند، و اطمینان می‌دهد که استفاده از داده‌ها با احترام به حریم خصوصی، شفاف و همسو با هدف نفع رساندن به جامعه است.

یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای کاوش داده‌ها برای منفعت اجتماعی با استفاده از یک مجموعه داده مرتبط با سلامت عمومی و پیش‌بینی بیماری:

```
In [27]: # آموزش: پیش‌بینی بیماری با استفاده از داده‌های سرطان پستان و مدل جنگل تصادفی

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 1 بارگذاری داده‌های سرطان پستان ویسکانسین از منبع UCI
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/breast-cancer-wisconsin/wdbc.data'
columns = [
    'id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean',
    'smoothness_mean', 'compactness_mean', 'concavity_mean', 'concave_points_mean',
    'symmetry_mean', 'fractal_dimension_mean'
]
data = pd.read_csv(url, names=columns, header=None)

# 2 پیش‌پردازش داده‌ها: حذف شناسه و برچسب‌گذاری کلاس‌ها
X = data.drop(['id', 'diagnosis'], axis=1)
y = (data['diagnosis'] == 'M').astype(int) # 'M' به 1 و 'B' به 0 تبدیل

# 3 تقسیم داده‌ها به آموزش و تست
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4 آموزش مدل جنگل تصادفی برای پیش‌بینی بیماری
model = RandomForestClassifier()
model.fit(X_train, y_train)

# 5 پیش‌بینی روی داده‌های تست
predictions = model.predict(X_test)

# 6 ارزیابی دقت مدل
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy for Disease Prediction: {accuracy:.4f}")
```

Accuracy for Disease Prediction: 1.0000

پیش‌بینی بیماری با داده‌های سرطان پستان و مدل جنگل تصادفی 🌿🧬

در این پروژه، داده‌های سرطان پستان ویسکانسین از منبع UCI برای آموزش یک مدل طبقه‌بندی استفاده شده‌اند. هدف، پیش‌بینی نوع تومور (خوش‌خیم یا بدخیم) با استفاده از ویژگی‌های آماری تصویر سلولی است. مراحل انجام‌شده به شرح زیر است:

- داده‌ها شامل ویژگی‌هایی مانند شعاع، بافت، محیط و تقعر سلول‌ها هستند.

- برچسب `diagnosis` به صورت عددی تبدیل شده تا مدل قابل آموزش باشد.
- مدل `RandomForestClassifier` روی داده‌های آموزشی آموزش داده شده و سپس روی داده‌های تست ارزیابی شده است.
- دقت مدل با استفاده از معیار `accuracy_score` محاسبه شده و نمایش داده شده است.

این نوع تحلیل برای ساخت سیستم‌های هوشمند تشخیص پزشکی، غربالگری اولیه، و پشتیبانی تصمیم‌گیری بالینی بسیار کاربردی است. می‌توان آن را با تحلیل ویژگی‌های مهم، ارزیابی حساسیت، یا مدل‌های پیشرفته‌تر مانند XGBoost یا شبکه‌های عصبی ترکیب کرد.

The End of Chapter 12